# Guía de estudio del Proyecto Integrador

Contrataciones públicas del Perú en formato OCDS.
Minería de Datos Masivos y Flujos de Datos.


1. Ricardo Amiel Acuña Villogas
2. Juan Leibniz Aquino Espinoza
3. Camilo Ernesto Soto Cristóbal


Este documento reúne las cinco guías de estudio de los notebooks y el glosario del dato. Explica qué hace cada paso, cómo funciona por dentro cada técnica, qué técnicas se descartaron y por qué, y cierra cada parte con las preguntas previsibles y su respuesta.

# Índice

**Notebook 1. KDD, análisis exploratorio y procesamiento distribuido** **5**

1. Qué responde esta parte . . . . . 5
2. El dataset . . . . . 6
3. Entradas y salidas exactas . . . . . 7
4. El proceso KDD, que es el esqueleto del notebook 7
5. Recorrido sección por sección . . . . . 8
6. Alternativas para cada decisión, explicadas . . 31
7. Preguntas probables y qué responder . . . . . 36

**Notebook 2. Similitud: shingling, Jaccard, MinHash, LSH y SimHash** **37**

1. Qué responde esta parte . . . . . 37
2. El esquema completo, en una frase . . . . . 37
3. Recorrido sección por sección . . . . . 38
4. Alternativas para cada decisión, explicadas . . 51
5. Preguntas probables y qué responder . . . . . 55

**Notebook 3. Búsqueda aproximada de vecinos: IVF y HNSW** **57**

1. Qué responde esta parte . . . . . 57
2. Entradas y salidas . . . . . 58
3. Recorrido sección por sección . . . . . 58

4. Alternativas para cada decisión, explicadas . . 67
5. Preguntas probables y qué responder . . . . . 70

**Notebook 4. Minería de flujos: muestreo, Bloom Filter, Count-Min Sketch y DGIM 72**

1. La restricción que da sentido a todo . . . . . 72
2. Recorrido sección por sección . . . . . 73
3. Por qué cada estructura y no la otra . . . . . 87
4. Técnicas del temario que no se implementaron, y por qué . . . . . 89
5. Preguntas probables y qué responder . . . . . 91

**Notebook 5. Conjuntos frecuentes y reglas de asociación 93**

1. Qué responde esta parte . . . . . 93
2. Qué es una entidad y por qué una contratación tiene canastas . . . . . 93
3. El vocabulario mínimo . . . . . 95
4. Recorrido sección por sección . . . . . 96
5. Alternativas para cada decisión, explicadas . . 106
6. Preguntas probables y qué responder . . . . . 111

**Glosario del dato 113**

1. De dónde viene el dato . . . . . 113
2. Los actores, que es la confusión más común . . 114

3. El ciclo de un proceso y qué significa que falte algo . . . . . 115
4. Los campos de la tabla analítica . . . . . 116
5. El código CUBSO, dígito por dígito . . . . . 117
6. Los métodos de contratación . . . . . 119
7. Los niveles de gobierno . . . . . 121
8. La categoría y el tramo de monto . . . . . 122
9. Las marcas y los indicadores derivados . . . . . 123
10. Confusiones frecuentes y cómo responderlas . 124
11. Cifras que conviene tener en la cabeza . . . . 126

# Notebook 1. KDD, análisis exploratorio y procesamiento distribuido

Guía de estudio del notebook 1_kdd_eda.ipynb. Explica qué hace cada sección, cómo funciona por dentro cada técnica, qué dice cada figura y qué decisiones se tomaron y se descartaron. El porqué más profundo de cada decisión está en docs/JUSTIFICACION.md y el guion de sustentación de cada figura en docs/FIGURAS.md.

Qué significa cada campo, cada código CUBSO y cada etiqueta del dato está en docs/GLOSARIO.md.

## 1. Qué responde esta parte

El enunciado pide aplicar el proceso KDD completo y demostrar procesamiento distribuido. Este notebook hace las dos cosas y, de paso, produce la tabla de la que viven las otras cuatro partes.

Las tres preguntas de interés se fijan antes de mirar los datos, que es lo que distingue un análisis de una pesca de correlaciones.

1. Qué tan concentrado está el gasto en pocos proveedores, y si eso cambia por territorio.
2. Qué relación hay entre el número de postores y el monto adjudicado.
3. Si hay apresuramiento del gasto en el cierre del ejercicio presupuestal.

## 2. El dataset

Contrataciones Abiertas del Perú en el estándar **OCDS**, Open Contracting Data Standard, publicadas por el OECE, antes OSCE. Se descargan del Registro de Datos de Open Contracting Partnership, publicación 135, paquetes anuales 2023, 2024 y 2025.

Hay que saber explicar tres cosas del formato.

**Qué es OCDS.** Es un estándar internacional que describe una contratación pública como un proceso con etapas: planificación, licitación, adjudicación, contrato y ejecución. Cada proceso tiene un identificador único y estable llamado **ocid**. El paquete descargado trae 21 tablas en CSV que son la versión aplanada de ese documento JSON anidado: main, awards, awards_suppliers, awards_items, parties, contracts, tender_items y demás.

**Qué es el grano del dato.** Las 21 tablas tienen granos distintos. En main hay una fila por proceso; en awards_items hay una fila por ítem adjudicado, de modo que un proceso puede aparecer en veinte filas. Mezclar tablas sin cuidar el grano duplica montos, y ese es el error silencioso más común con estos datos.

**Qué es el código CUBSO.** Es el Catálogo Único de Bienes, Servicios y Obras del Estado peruano, derivado de la codificación UNSPSC. Tiene 16 dígitos: los 8 primeros ubican el bien o servicio en la jerarquía del catálogo y los 8 últimos son el correlativo del ítem concreto. El proyecto usa el prefijo de 8, porque el código completo es casi único por ítem y daría un universo tan disperso que ningún conjunto alcanzaría el soporte mínimo en la Parte V.

Volumen: 2,98 GB de CSV, 281.462 procesos crudos, 177.398 tras la limpieza, 29,8 MB en Parquet.

## 3. Entradas y salidas exactas

**Entrada.** Los tres paquetes anuales. La función **asegurar_datos** de notebooks/scripts/ingesta.py los descarga si faltan, de modo que el notebook corre de cero en otra máquina sin pasos manuales.

**Salida.** La tabla analítica en Parquet particionado por año, en data/processed, más 18 artefactos en CSV y JSON y 15 figuras en SVG y PNG.

## 4. El proceso KDD, que es el esqueleto del notebook

KDD, Knowledge Discovery in Databases, tiene cinco etapas y el notebook está ordenado por ellas. Conviene saber diferenciarlo de **CRISP DM**, que es un modelo de proceso de negocio con seis fases y un bucle explícito; KDD es más estrecho y se centra en el camino del dato.

| Etapa de KDD | Dónde está en el notebook | Qué produce |
|---|---|---|
| Selección | Sección 1 | 5 tablas de 21, desnormalizadas a una fila por proceso |
| Preprocesamiento | Secciones 2 y 3 | Perfil de calidad y seis decisiones de limpieza con bitácora |
| Transformación | Sección 4 | Variables derivadas y Parquet particionado por año |
| Minería | Sección 5 | El análisis exploratorio, y las Partes II a V |
| Interpretación | Sección 8 y Parte VI | Las lecturas de cada figura y el reporte |

## 5. Recorrido sección por sección

### Sección 1. Selección

De las 21 tablas se usan 5 y se unen a una fila por proceso.

1. **main.csv** da el proceso: ocid, fecha de convocatoria, objeto, categoría, método, comprador, monto referencial en soles y número de postores.
2. **awards.csv** y **awards_suppliers.csv** dan el proveedor adjudicado. Un proceso puede tener varias adjudicaciones, así que se usa una ventana **row_number** ordenada por monto adjudicado descendente para quedarse con el proveedor de la mayor, y aparte se cuenta cuántos proveedores distintos hubo, que es una señal de fraccionamiento.
3. **awards_items.csv** da la canasta de códigos CUBSO del proceso, recortados a 8 dígitos y agregados con

**collect_set**.

4. **parties.csv** da el departamento del comprador, filtrando por el rol buyer.

Dos detalles que hay que saber defender.

**Todo se lee como texto, con inferSchema en falso.** Si Spark infiriera el esquema, el código CUBSO de 16 dígitos pasaría a coma flotante y volvería en notación científica, perdiendo justo los dígitos que identifican el ítem. Los campos numéricos se convierten después, uno a uno y a propósito.

**La fecha es tender_datePublished y no publishedDate.** La primera es la fecha del evento, cuándo se convocó; la segunda es la fecha del registro, cuándo entró al paquete. Usar la del registro describiría la actividad del publicador y no la del Estado comprador.

## Sección 2. Perfilado y calidad

Antes de decidir nada se mide. El perfil cuenta nulos y cardinalidad de cada columna.

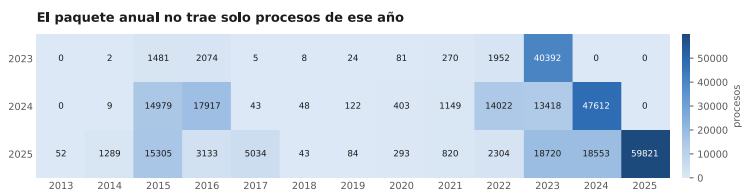
La cardinalidad se calcula con **approx_count_distinct**, que es **HyperLogLog**, con error relativo del 2 por ciento. Conviene saber por qué: contar distintos de forma exacta obliga a mantener en memoria todos los valores vistos, mientras que HyperLogLog guarda un arreglo de registros pequeños y estima la cardinalidad a partir del máximo número de ceros iniciales observado en el hash de cada valor. La intuición es que ver un hash con muchos ceros iniciales es improbable, así que su aparición sugiere que se han visto muchos elementos distintos. Para esta tarea

solo hace falta el orden de magnitud, lo justo para distinguir un identificador de una categórica y de una continua, de modo que pagar un conteo exacto sería desperdicio.

El perfil deja ver tres cosas.

1. El monto no tiene ni un solo nulo, pero el valor faltante viene codificado como cero: 22.830 filas.
2. El proveedor falta en uno de cada cinco procesos, 61.807 filas, que son los desiertos, anulados o en trámite.
3. El año muestra trece valores distintos pese a que se bajaron tres paquetes anuales.

Esa tercera anomalía se investiga con una tabla cruzada de año del paquete contra año de convocatoria, que produce la **figura f01**. Si cada paquete trajera solo sus procesos, toda la masa estaría en la diagonal. No lo está: 82.946 filas son convocatorias anteriores a 2023, y en el paquete 2024 casi la mitad son de 2015 y 2016. Es el hallazgo que justifica recortar la ventana temporal.




**f01.** Los paquetes anuales rellenan años anteriores

**Sección 3. Limpieza**

Seis decisiones. La política es que eliminar es la última opción y que cada decisión deja constancia: la función **limpiar** devuelve la tabla y una bitácora con cuántas filas tocó cada paso.

| Paso | Qué hace | Filas afectadas | Quedan |
|---|---|---|---|
| entrada | tabla analítica cruda | 0 | 281.462 |
| duplica-dos | una fila por ocid, la más reciente | 0 | 281.462 |
| descarte | monto nulo o no positivo | 22.830 | 258.632 |
| descarte | fecha fuera de 2023 a 2025 | 81.234 | 177.398 |
| impu-tación | postores con la mediana de su método | 6.655 | 177.398 |
| marca | monto extremo en escala logarítmica, se conserva | 565 | 177.398 |
| recorte | postores winsorizados al percentil 99,9, tope 107 | 263 | 177.398 |

Cómo defender cada una.

1. **Duplicados.** Ventana por ocid ordenada por fecha de registro descendente, se conserva la versión registrada más tarde. No elimina nada porque los ocid ya venían únicos, pero el paso se deja porque la unión de tres paquetes no garantiza esa unicidad.
2. **Monto no positivo.** El cero no es un monto bajo, es un campo no publicado. No admite imputación: poner la mediana inventaría gasto público que no existe y hundiría cualquier estimación agregada.
3. **Ventana temporal.** Es la consecuencia de la figura

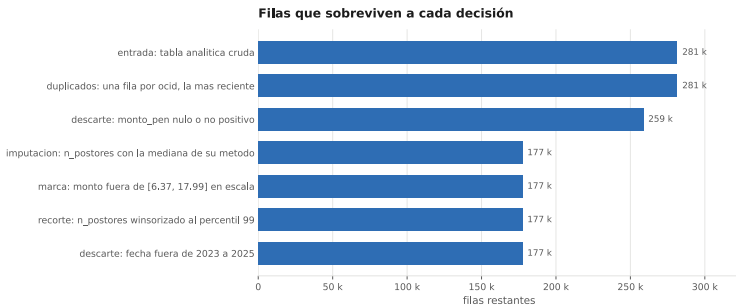
f01.

4. **Postores faltantes.** La ausencia no es al azar, se concentra en ciertos métodos, así que la mediana se calcula **dentro de cada método** y solo se cae a la mediana global si el método no tiene ninguna observación. Y se añade una columna booleana **n_postores_imputado**, de modo que todo análisis de competencia pueda excluir las filas imputadas y la relación no la produzca la propia imputación.
5. **Extremos de monto.** Se marcan y se conservan. El criterio es el intervalo intercuartílico pero aplicado sobre el logaritmo del monto y con factor 3 en lugar de 1,5, porque la distribución es de cola pesada y en escala lineal el criterio clásico marcaría como extremo una fracción enorme. El resultado es que 565 procesos, el 0,3 por ciento de las filas, concentran el 38 por ciento del gasto. Eliminarlos habría dejado una tabla más ordenada y habría borrado un tercio del dinero público que se quiere estudiar.
6. **Postores absurdos.** Aquí el extremo sí es error de carga y no tiene lectura sustantiva, así que se **winsoriza**, es decir se recorta al percentil 99,9, tope 107. Winsorizar no es eliminar: la fila sigue aportando monto, objeto y canasta.

Además hay una reparación de codificación. El SEACE grabó parte de los textos en CP850 y el registro los leyó como Latin-1, de modo que cada letra acentuada quedó convertida siempre en el mismo carácter equivocado. Once sustituciones deshacen esa conversión. Lo ambiguo se deja sin tocar, sobre todo el signo de interrogación invertido, que unas veces sustituye una tilde

y otras está puesto a propósito, y se cuenta cuántas filas lo llevan para que el lector sepa el tamaño del residuo. La celda de verificación comprueba que los caracteres reparables quedan en cero.

La **figura f02** es la bitácora hecha gráfico: se pierden 104.064 filas en total, casi todas en dos pasos, y los demás pasos imputan, marcan o recortan sin perder ninguna.




**f02.** El embudo de limpieza con su costo en filas

Una columna resultó inservible y se documenta como tal: la duración de la convocatoria daba cero en todas las filas, porque la fecha de cierre de la licitación viene vacía en este paquete.

## Sección 4. Transformación y persistencia

Las variables derivadas son el año, el mes, el año y mes, y el nivel de gobierno.

El **nivel de gobierno** no viene en el dato crudo: se deriva con un CASE de expresiones regulares sobre el nombre

de la entidad, con reglas para municipalidades, gobiernos regionales, universidades, establecimientos de salud, unidades educativas, fuerzas armadas y policiales, empresas del Estado y gobierno nacional. Es una heurística, y hay que presentarla como tal. La categoría Otro recoge lo que ninguna regla reconoce y es el 6,1 por ciento de los procesos; esa cifra es la medida honesta de cuánto falla la derivación y por eso se reporta.

La tabla se guarda en **Parquet particionado por año**. Dos propiedades que conviene saber explicar.

1. **Parquet es columnar.** Guarda los valores de una columna juntos y comprimidos, con estadísticas por grupo de filas. El análisis exploratorio lee dos o tres columnas de veinticinco en vez de la fila entera, y la compresión es mucho mejor porque los valores de una misma columna se parecen entre sí. De ahí que 2,98 GB de CSV queden en 29,8 MB.
2. **El particionado crea directorios.** Al escribir **partitionBy** por año Spark crea carpetas anio=2023, anio=2024 y anio=2025, y el año deja de almacenarse en los archivos: se lee del nombre de la carpeta.

La columna de partición se elige por cómo se consulta la tabla. El año es la correcta porque toda la Parte IV la recorre en orden temporal. Particionar por departamento habría dado veinticinco directorios muy desiguales, con Lima concentrando casi todo, y esa asimetría se paga en cada barajado.

## Sección 5. El análisis exploratorio, figura por figura

Todo el cálculo ocurre en Spark y a pandas solo llega el agregado ya reducido. Esa es la regla que hace que el EDA sea distribuido de verdad y no un **toPandas** disfrazado.

**f03, procesos por mes.** Diciembre es el máximo de su año en los dos años con cobertura completa: 7.280 procesos en 2023 y 6.498 en 2024. La cola de 2025 cae a 3.380 en noviembre y 942 en diciembre. Eso no es menos contratación sino subcobertura, porque los procesos tardan en entrar al registro OCDS. Es un sesgo de publicación y hay que decirlo, porque un lector apresurado lo leería como una caída del gasto.




**f03.** Procesos por mes y el límite de cobertura de 2025

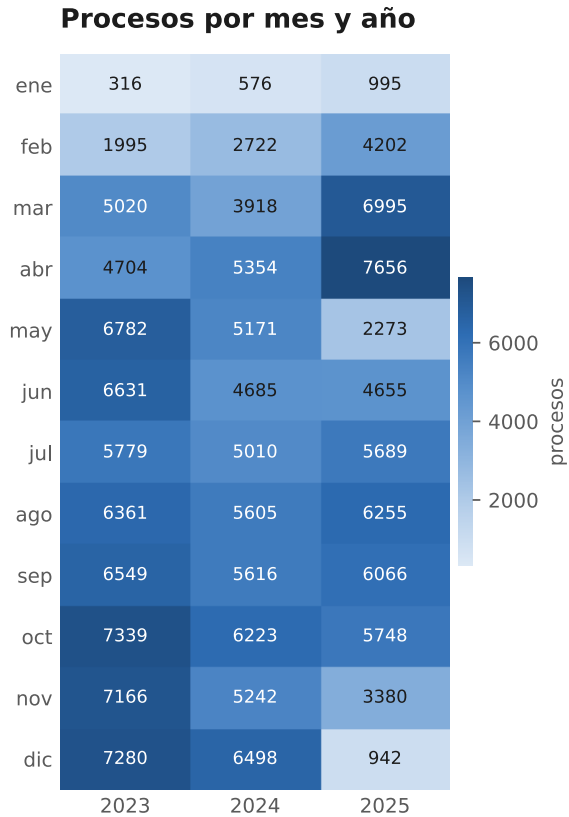
**f04, monto por mes.** Se grafica aparte del conteo a propósito. Con dos series en un eje doble el lector lee una correlación que la figura no sostiene, y el eje doble

permite elegir escalas que hagan coincidir dos curvas que no tienen por qué coincidir. El máximo de toda la serie es diciembre de 2024 con 14.373 millones de soles, 1,9 veces el mes anterior: a fin de año no solo se convoca más, se convoca más caro.




**f04.** Monto por mes, en escala propia

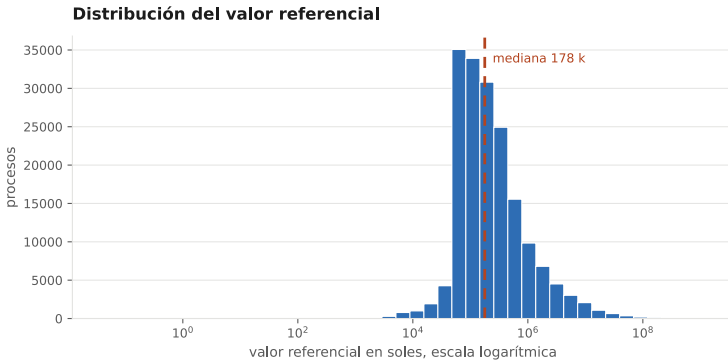
**f05, estacionalidad.** Es un mapa de calor de mes contra año y es la figura con la que se responde la pregunta 3. Aísla el efecto calendario del efecto año. Que el patrón del último trimestre se repita en 2023 y en 2024 es la evidencia mínima para atribuirlo al cierre del ejercicio presupuestal y no a un año particular. Con un solo año no se podría afirmar.




**f05.** Estacionalidad del gasto, mes contra año

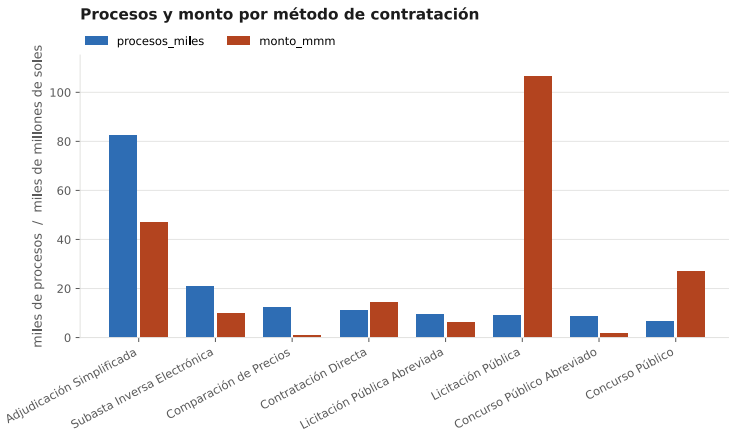
**f06, distribución del monto.** Histograma en escala logarítmica, con la mediana marcada. La media es 7,9 veces la mediana, así que la media no describe a ningún proceso representativo. En escala lineal esta figura sería una sola barra pegada al eje; el logaritmo es lo que la hace legible y es la misma escala sobre la que se aplicó el criterio de extremos, de modo que el criterio y su figura

son coherentes.



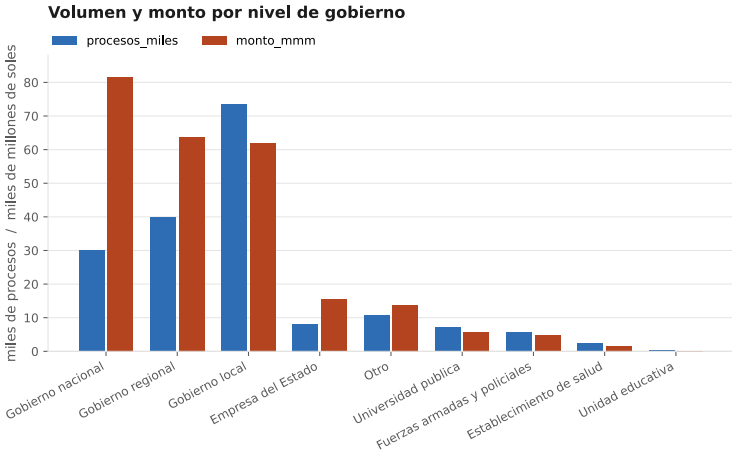

**f06.** Distribución del monto en escala logarítmica

**f07, método de contratación.** Contar procesos y contar soles dan rankings casi inversos. Adjudicación Simplificada encabeza por volumen con 82.503 procesos pero mueve 47,1 mil millones; Licitación Pública, con 9.242 procesos, mueve 106,6 mil millones, el 46 por ciento del total. La consecuencia práctica es que toda afirmación debe decir en qué unidad está contando: la mitad de las lecturas erróneas de datos de contratación vienen de mezclar las dos.



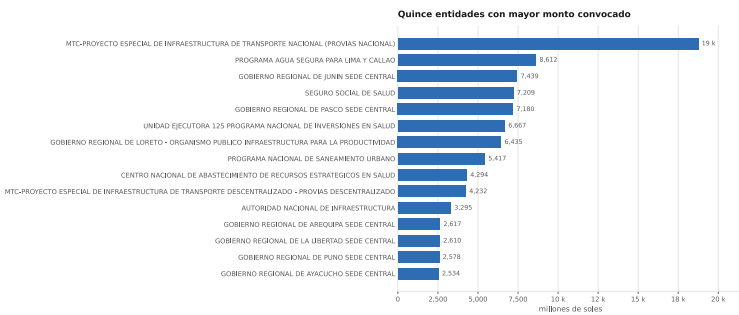

**f07.** Método de contratación, por número y por monto

**f08, nivel de gobierno.** Los gobiernos locales convocan 73.456 procesos repartidos en 1.855 entidades y el gobierno nacional los supera en monto con 29.984 procesos y solo 277 entidades.




**f08.** Nivel de gobierno, una variable derivada

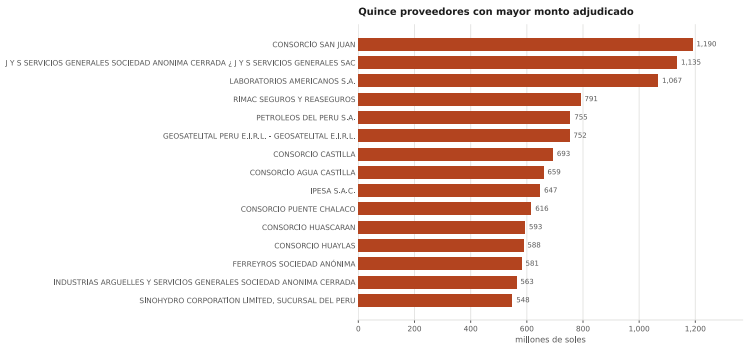
**f09, los quince compradores con mayor monto.** Se ordena por monto y no por número de procesos porque la pregunta es a dónde va el dinero, no quién hace más trámites. Suman 89,9 mil millones de soles.




**f09.** Las quince entidades que más monto concentran

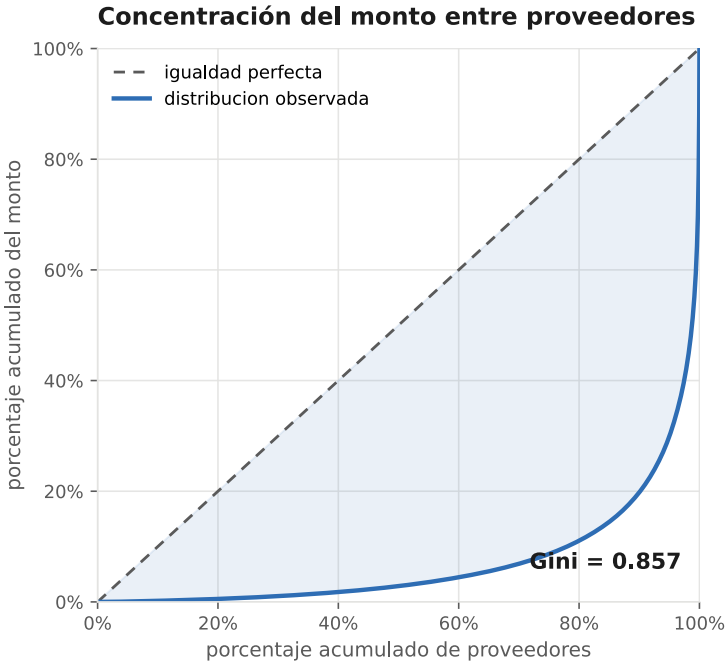
**f10, los quince proveedores con mayor monto.** Aquí

está el hallazgo que abre la Parte II: hay dos formas distintas de llegar arriba y el ranking las pone una al lado de la otra sin distinguirlas. Un proveedor acumula 581 millones repartidos en 188 adjudicaciones; otro alcanza 1.135 millones en una sola. Recurrencia y magnitud puntual son fenómenos que no se parecen.




**f10.** Los quince proveedores que más monto reciben

**f11, curva de Lorenz y coeficiente de Gini.** Esta es la respuesta cuantitativa a la pregunta 1.



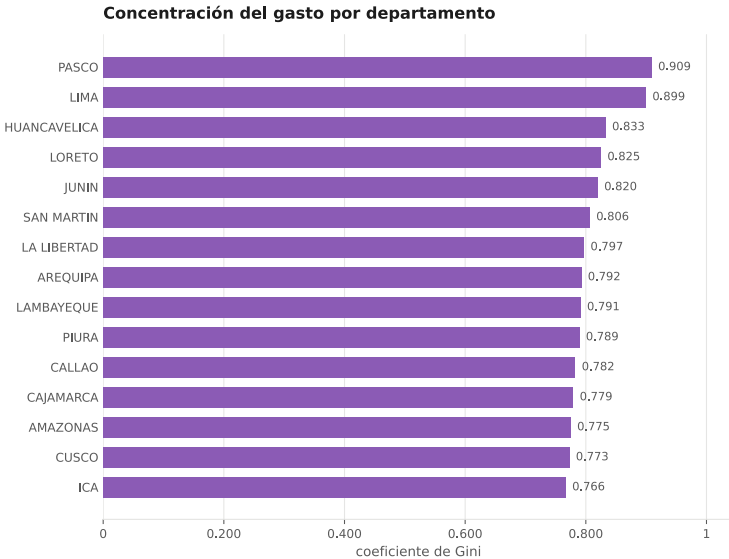

**f11.** Curva de Lorenz y coeficiente de Gini de proveedores

Cómo funciona. Se ordenan los proveedores de menor a mayor monto adjudicado y se dibuja, en el eje horizontal, la fracción acumulada de proveedores y en el vertical la fracción acumulada del monto. Si el reparto fuera perfecto la curva sería la diagonal. El **Gini** es el doble del área entre la diagonal y la curva, y va de 0, reparto perfecto, a 1, un solo proveedor se lleva todo.

El resultado es Gini 0,857 sobre 67.510 proveedores. El uno por ciento superior se lleva el 47,1 por ciento del monto, el cinco por ciento el 70,1, el diez por ciento el 80,2 y el veinticinco por ciento el 91,3. Para dar

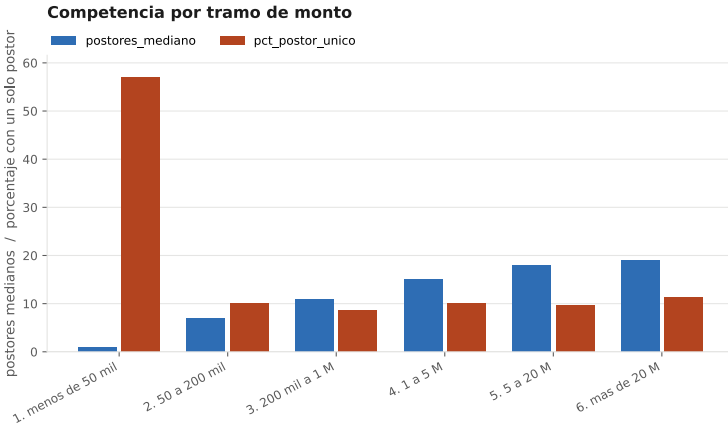
referencia: los Gini de ingreso de los países suelen estar entre 0,25 y 0,60, así que 0,857 es concentración alta. Lo que no permite afirmar es que la causa sea anticompetitiva: puede ser economía de escala, especialización técnica o barreras de entrada.

**f12, Gini por departamento.** Esta figura corrige lo que la anterior podría hacer creer. El Gini es adimensional, así que hace comparables mercados de tamaños muy distintos, cosa que el monto absoluto no permite. El resultado va en contra de lo esperado: solo 2 de 25 departamentos superan el Gini nacional, Pasco con 0,909 y Lima con 0,899. El resto queda por debajo. La cifra del país no describe a un mercado regional típico, está empujada hacia arriba por Lima. Se exige un mínimo de 50 proveedores por departamento para que el Gini sea estable.




**f12.** Gini por departamento contra el nacional

**f13, competencia por tramo de monto.** Responde la pregunta 2 y el hallazgo es que hay dos fenómenos y no uno.




**f13.** Número de postores por tramo de monto

| Tramo | Procesos | Postores medianos | Con un solo postor |
|---|---|---|---|
| menos de 50 mil | 10.156 | 1 | 57,0 % |
| 50 a 200 mil | 83.628 | 7 | 10,1 % |
| 200 mil a 1 M | 55.872 | 11 | 8,7 % |
| 1 a 5 M | 15.931 | 15 | 10,0 % |
| 5 a 20 M | 3.998 | 18 | 9,7 % |
| más de 20 M | 1.158 | 19 | 11,4 % |

El número mediano de postores sí crece con el monto. Lo que importa es dónde se rompe: debajo de cincuenta mil soles el 57 por ciento tuvo un único postor, y en los demás tramos la proporción se estabiliza cerca del 10 por ciento y ya no baja ni por encima de veinte millones. Son dos cosas distintas, una franja pequeña competitiva solo en el papel y un suelo persistente de uno de cada diez

procesos sin competencia. Ese suelo es la pregunta que recogen la Parte IV y la Parte V.

Se agrupa por tramos en lugar de graficar los puntos porque 170.743 puntos son una mancha negra sin estructura legible, y se excluyen los procesos con postores imputados.

### Sección 6. MapReduce sobre RDD frente a DataFrames y Spark SQL

Se implementan dos agregaciones en los dos estilos, cada una ligada a una pregunta: el monto por departamento alimenta la pregunta 1 y los postores por método alimentan la pregunta 2.

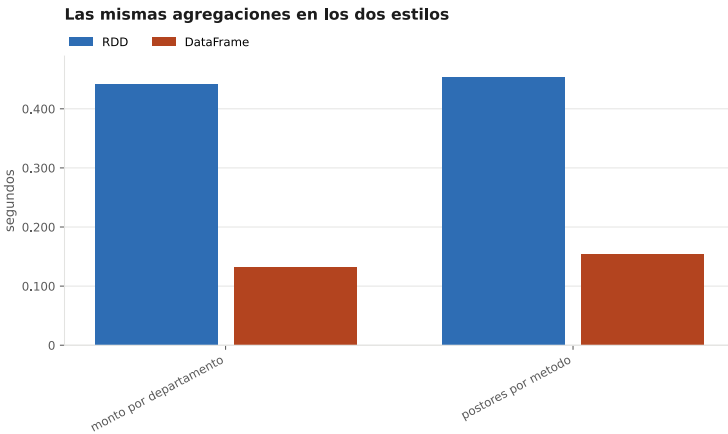
**Cómo se escribe el estilo MapReduce.** Se mapea cada fila a un par clave y valor, donde el valor lleva la suma parcial y el conteo, y se combina con **reduceByKey**. Hay que saber por qué se usa **reduceByKey** y no **groupByKey**: el primero aplica la función de combinación **dentro de cada partición antes de barajar**, exactamente como el combiner de MapReduce clásico, de modo que por la red solo viaja un par por clave y partición. Con **groupByKey** viajarían todos los valores.

**Cómo se verifica que dan lo mismo.** No se comparan sumas, que es fácil de hacer coincidir por casualidad, sino los conjuntos completos con **exceptAll** en los dos sentidos, que detecta filas sobrantes y faltantes.

**Cómo se mide el tiempo.** Con calentamiento previo y mediana de tres corridas, porque una sola corrida no es una medición: la primera paga la carga de clases de la

máquina virtual de Java y la caché del sistema de archivos.

El resultado, en la **figura f14**, es que el camino RDD tarda entre 3,0 y 3,4 veces más. No es por gusto.




**f14.** El mismo agregado por RDD y por DataFrame

| Aspecto | MapReduce sobre RDD | DataFrame y Spark SQL |
|---|---|---|
| Qué se escribe | Cómo transformar cada fila y cómo combinar los parciales | Qué resultado se quiere; el planificador decide cómo |
| Esquema | Spark ve tuplas opacas y no valida nombres ni tipos hasta ejecutar | Un error de columna se detecta al planificar |
| Optimización | La que escriba a mano el programador | Catalyst poda columnas, reordena filtros y elige la unión |
| Serialización | Cada fila cruza a Python, y ese es el costo dominante | El cómputo se queda en la máquina virtual de Java |
| Cuándo conviene igual | Estado entre elementos, estructuras propias, algoritmos iterativos | Seleccionar, filtrar, agrupar, unir y agregar |

Las dos causas de la brecha son la frontera entre la máquina virtual de Java y el intérprete de Python, que cada fila tiene que cruzar en los dos sentidos, y que el planificador no puede podar columnas ni reordenar filtros sobre tuplas opacas.

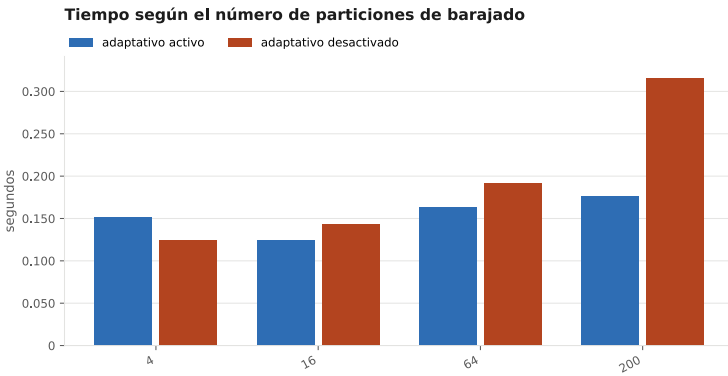
Si preguntan por qué implementar el RDD si el DataFrame siempre gana: porque la rúbrica pide demostrar MapReduce, y escribirlo obliga a declarar la clave, el valor y la función de reducción, que es justamente lo que Catalyst esconde; y porque el RDD sigue siendo la herramienta cuando la operación no se expresa en la API de DataFrames, que es lo que ocurre en las firmas MinHash de la Parte II.

## Sección 7. Particionado y plan de ejecución

Una partición es la unidad de paralelismo: Spark lanza una tarea por partición. Dos cosas se verifican leyendo el plan con **explain** en modo formateado en lugar de suponerlas.

1. Un filtro sobre la columna de partición aparece en **PartitionFilters**. Spark lo resuelve en el catálogo de archivos antes de leer un byte y abre un directorio de tres.
2. Un filtro sobre una columna no particionada aparece en **PushedFilters**. Spark abre los tres directorios pero usa las estadísticas por grupo de filas de Parquet, mínimo y máximo de cada columna en cada bloque, para saltar bloques sin descomprimirlos. Es una optimización real pero de segundo orden frente a no abrir el archivo.

La **figura f15** mide el tiempo de una agregación con 4, 16, 64 y 200 particiones de barajado, dos veces: con ejecución adaptativa y sin ella.




**f15.** Tiempo según el número de particiones de barajado

Con **ejecución adaptativa**, que es la configuración real, el parámetro casi no cambia el tiempo, porque Spark conoce el tamaño real de la salida del barajado y fusiona las particiones pequeñas, de modo que doscientas declaradas terminan siendo unas pocas efectivas. Al desactivarla aparece el efecto que predice la teoría: 0,32 segundos con 200 particiones frente a 0,14 con 16, porque planificar y recoger doscientas tareas de unos cientos de filas cuesta más que el cálculo.

La conclusión matizada, que es la que hay que decir: la regla de dos a cuatro veces el número de núcleos sigue siendo el punto de partida razonable, y la ejecución adaptativa protege contra declarar demasiadas particiones pero no contra declarar muy pocas, porque fusiona las pequeñas y no puede partir las grandes.

**Sección 8. Qué hereda cada parte**

La tabla persistida deja listas tres representaciones que no hay que volver a construir.

1. El objeto contractual normalizado y con la codificación reparada, del que salen los shingles de la Parte II y los vectores TF-IDF de la Parte III.
2. El conjunto de códigos CUBSO por proceso, entrada de Jaccard y MinHash en la Parte II y base de la canasta de la Parte V.
3. La fecha de convocatoria con resolución de minuto, que ordena el flujo simulado de la Parte IV.

Y deja dos preguntas abiertas que otras partes recogen: si los proveedores distintos del ranking son en realidad la misma empresa, que es la Parte II, y en qué combinaciones de entidad, rubro y método se concentra el suelo de procesos sin competencia, que es la Parte V y, sobre una ventana reciente, la Parte IV.

## 6. Alternativas para cada decisión, explicadas

Cada decisión de esta parte tenía al menos otra opción razonable. Saber nombrarla, explicar qué hace y por qué se descartó es lo que distingue una decisión de una costumbre.

**Para los valores faltantes**

1. **Imputar el monto cero con la mediana.** Es lo que se haría por defecto. Se descarta porque el cero no es un valor bajo sino un campo no publicado: imputarlo inventaría gasto público que no existe y, como la Parte I estima montos agregados, ese invento se propagaría a todas las conclusiones.
2. **Imputación por vecinos más cercanos.** Rellena con el promedio de las k filas más parecidas en las demás columnas. Es defendible para el número de postores, pero añade una estructura que después se usa como resultado: la Parte I analiza precisamente la relación entre postores y monto, de modo que imputar postores a partir del monto crearía la correlación que se quiere medir.
3. **Imputación múltiple por ecuaciones encadenadas.** Genera varias versiones del dataset y propaga la incertidumbre. Es lo correcto en inferencia estadística, y es desproporcionado aquí: el entregable no es un intervalo de confianza sino un análisis exploratorio.
4. **Lo que se hizo.** Mediana **condicional al método**, porque la ausencia no es al azar y se concentra en ciertos métodos, más una columna booleana que marca la fila como imputada para poder excluirla. Esa columna es la clave: permite que la figura f13 se calcule solo sobre datos observados.

**Para los valores extremos**

1. **Puntuación z, es decir más de tres desviaciones típicas.** Supone normalidad. Con una distribución de cola pesada como el monto, la desviación típica está dominada por los propios extremos y el criterio deja de discriminar.
2. **Criterio de Tukey clásico, 1,5 veces el rango intercuartílico, en escala lineal.** Marcaría como extrema una fracción enorme de las filas, porque la distribución es asimétrica por naturaleza.
3. **Recorte por percentil, por ejemplo al 99.** Es arbitrario y además elimina.
4. **Bosque de aislamiento.** Detección multivariante de anomalías. Resuelve otro problema: aquí no se busca una fila rara en varias dimensiones, sino un monto alto, que es univariante y además es el objeto de estudio.
5. **Lo que se hizo.** Tukey con factor 3 sobre el logaritmo del monto, y **marcar en lugar de eliminar**. La escala logarítmica es la que hace simétrica la distribución, y el factor 3 en lugar de 1,5 reconoce que la cola es real. El resultado es que los 565 marcados concentran el 38 por ciento del gasto, que es justamente la razón para no borrarlos.

**Para el perfilado de cardinalidad**

1. **Conteo exacto de distintos.** Obliga a mantener todos los valores vistos. Sobre 25 columnas y 281.462 filas es un gasto innecesario para una decisión que solo necesita el orden de magnitud.

2. **Flajolet y Martin.** El antecesor de HyperLogLog: estima la cardinalidad a partir del máximo número de ceros iniciales en los hashes de los elementos. Tiene varianza alta con un solo estimador.
3. **Lo que se hizo.** HyperLogLog, que es Flajolet y Martin promediando muchos registros con media armónica y corrigiendo el sesgo, con error relativo fijado en el 2 por ciento.

**Para la duplicación**

1. **Desduplicación difusa por nombre.** Dos proveedores escritos distinto que son la misma empresa. No se hace aquí a propósito: es precisamente el problema de la Parte II, y resolverlo con una regla de texto antes de medirlo sería dar por supuesto lo que se quiere demostrar.
2. **Lo que se hizo.** Desduplicación exacta por ocid, conservando la versión registrada más tarde.

**Para el modelo de proceso**

1. **CRISP-DM.** Seis fases, con comprensión del negocio al inicio y despliegue al final, y bucles explícitos. Es un modelo de proyecto.
2. **SEMMA.** Muestrear, explorar, modificar, modelar y evaluar. Está centrado en el modelado y da por hecha la fase de datos.
3. **Lo que se hizo.** KDD, porque es el que el curso usa y porque su foco es el camino del dato, que es lo que

esta parte entrega. Se menciona CRISP-DM porque la diferencia es una pregunta probable.

**Para el almacenamiento**

1. **CSV.** Lo que vino. Fila a fila, sin esquema y sin compresión por columna.
2. **ORC o Avro.** ORC es también columnar y comparable a Parquet; Avro es por filas, bueno para escribir flujos y malo para agregar columnas sueltas.
3. **Particionar por departamento.** Veinticinco directorios muy desiguales, con Lima concentrando casi todo. La asimetría se paga en cada barajado y ninguna parte recorre la tabla por territorio.
4. **No particionar.** Pierde la poda de directorios, que es lo que hace que un filtro por año no abra dos tercios de los archivos.
5. **Lo que se hizo.** Parquet particionado por año, porque toda la Parte IV recorre la tabla en orden temporal.

**Para el procesamiento**

1. **pandas sobre la máquina entera.** Cabría, con 177 mil filas. No demostraría lo que la rúbrica pide y no serviría al escenario de diez veces más volumen.
2. **Solo DataFrames, sin RDD.** Sería lo más rápido, pero el RDD es donde se ve MapReduce con su clave, su valor y su función de reducción, que es justo lo que Catalyst esconde.

3. **Lo que se hizo.** Las dos, con la misma agregación, verificando que coinciden y midiendo la diferencia.

## 7. Preguntas probables y qué responder

**¿Por qué eliminar el 37 por ciento de las filas no es demasiado?** Porque está concentrado en dos decisiones defendibles, el monto no publicado y la ventana temporal, y los otros cinco pasos juntos no pierden ninguna fila.

**¿Cómo saben que la caída de 2025 es subcobertura y no una caída real del gasto?** Porque el patrón de los dos años completos es el inverso, con máximo en diciembre, y porque la caída empieza justo donde termina la ventana de publicación del registro.

**¿Qué confianza tiene el nivel de gobierno si es una expresión regular?** La medimos en lugar de suponerla: la categoría Otro es el 6,1 por ciento y esa cifra es la cota de error de la derivación.

**¿Un Gini de 0,857 es mucho?** Comparado con el caso teórico, cero es reparto perfecto y uno un solo proveedor; y comparado con los Gini de ingreso de los países, que suelen estar entre 0,25 y 0,60, es concentración alta. No permite afirmar conducta anticompetitiva.

**¿Qué aporta Spark aquí de verdad?** La ingesta y el análisis exploratorio, sin duda: 2,98 GB en 21 tablas por año se desnormalizan a una tabla analítica en menos de medio minuto. Y queda demostrado con medición, no con afirmación, dónde conviene cada API.

# Notebook 2. Similitud: shingling, Jaccard, MinHash, LSH y SimHash

Guía de estudio del notebook 2_similitud_lsh.ipynb. El porqué de cada elección está en docs/JUSTIFICACION.md, sección Parte II.

Qué significa cada campo, cada código CUBSO y cada etiqueta del dato está en docs/GLOSARIO.md.

## 1. Qué responde esta parte

La Parte I dejó dos cosas abiertas. Primero, hay procesos cuyo objeto contractual es prácticamente el mismo texto repetido entre entidades. Segundo, hay proveedores que aparecen como distintos siendo la misma empresa escrita de otra forma.

Ninguna de las dos se detecta comparando todos los pares: con 177.398 procesos son más de quince mil millones de comparaciones. El problema es cuadrático y la parte entera trata de convertirlo en uno aproximadamente lineal sin perder lo que importa.

Spark es obligatorio aquí. Las firmas y el agrupamiento por bandas corren sobre RDD, que es donde el estilo imperativo gana al declarativo.

## 2. El esquema completo, en una frase

Texto del objeto, pasado a **conjunto de k-shingles**, resumido en una **firma MinHash** de 128 enteros, partida

en **bandas** que generan candidatos, y los candidatos pasan por una **verificación exacta** de Jaccard. En paralelo, el mismo texto pasado a **TF-IDF** y resumido con **SimHash**, para comparar los dos esquemas de hashing sensible a la localidad.

## 3. Recorrido sección por sección

### Sección 1. Dos representaciones como conjunto

El dataset ofrece dos formas de ver un proceso como conjunto, y compararlas es en sí mismo un resultado.

| Representación | Elementos por proceso | Universo | Procesos vacíos |
|---|---|---|---|
| Shingles del objeto con k igual a 9 | 186,9 | 137.412 | 0 |
| Conjunto de códigos CUBSO | 0,94 | 654 | 270 de 2.000 |

La tabla decide. El conjunto CUBSO tiene menos de un elemento por proceso en promedio y 270 procesos vacíos, así que no sostiene una similitud discriminante: la Jaccard entre dos conjuntos de un elemento solo puede ser cero o uno. Los shingles del objeto sí la sostienen. El CUBSO se reserva para la Parte V, donde la canasta se redefine por entidad y mes.

**Qué es un k-shingle.** Es cada subcadena contigua de k caracteres del texto. El documento se representa por el **conjunto** de sus shingles, no por la lista: se pierde el orden global pero se conserva el orden local, porque cada

shingle es un fragmento contiguo. Dos textos con las mismas palabras en distinto orden comparten pocos shingles si las fronteras caen distinto, que es lo que se quiere.

**Por qué shingles de caracteres y no de palabras.** Porque el objeto lo escribe a mano cada entidad y abundan los errores de digitación, las abreviaturas y los plurales. Los shingles de caracteres toleran eso; los de palabras no.

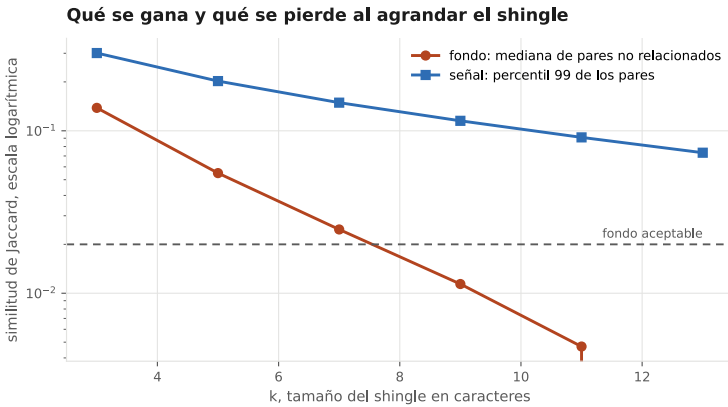
**La calibración de k.** Esta es la figura que más conviene saber defender porque documenta un resultado negativo.

| k | Shingles por documento | Fondo, mediana de pares no relacionados | Señal, percentil 99 |
|---|---|---|---|
| 3 | 153,7 | 0,1384 | 0,3010 |
| 5 | 179,4 | 0,0549 | 0,2023 |
| 7 | 186,0 | 0,0247 | 0,1490 |
| 9 | 187,1 | 0,0114 | 0,1152 |
| 11 | 186,5 | 0,0047 | 0,0911 |
| 13 | 185,3 | 0,0000 | 0,0733 |

Buscábamos un k óptimo y no existe: las dos curvas bajan juntas, así que ninguna métrica sobre pares aleatorios tiene un máximo interior, y construir una métrica que lo tuviera sería fabricar el resultado. Lo que se hace es **fijar un criterio explícito**: k debe ser lo bastante grande para que dos objetos no relacionados no compartan casi nada, en concreto que el fondo quede por debajo de 0,02. El primer k que lo cumple es 9.

Con k igual a 3 el fondo está en 0,138, es decir que dos textos cualesquiera ya se parecen un 14 por ciento solo

por ser español administrativo. Y la elección no es crítica: el barrido de LSH da resultados muy parecidos con 7 y con 11. Esta es la **figura f16**.




**f16.** Calibración del tamaño del shingle

## Sección 2. Jaccard exacta, la verdad de referencia

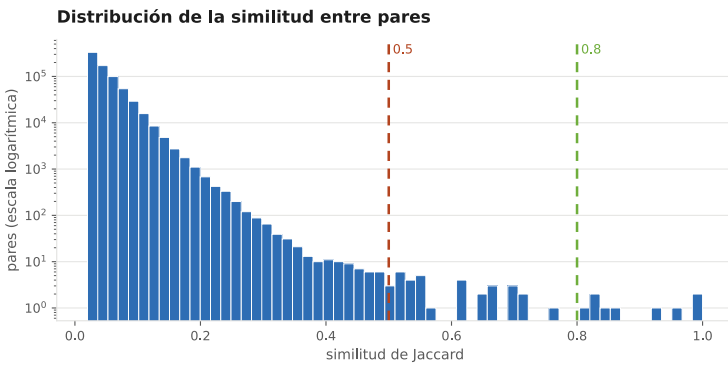
Sobre una muestra de 2.000 procesos se calculan los 1.999.000 pares. Esa muestra existe para una sola cosa: tener una verdad de referencia contra la que medir todo lo aproximado.

**Qué es Jaccard.** El tamaño de la intersección dividido entre el tamaño de la unión de los dos conjuntos. Va de 0, sin nada en común, a 1, idénticos.

**Cómo se calcula para dos millones de pares sin un bucle.** Con álgebra dispersa. Se arma una matriz binaria

M de documentos por shingles, donde la celda vale uno si el documento contiene ese shingle. Entonces el producto M por M traspuesta da, en cada celda, el **tamaño de la intersección** de ese par. La unión sale de la identidad de inclusión y exclusión: tamaño de A más tamaño de B menos la intersección. Es una sola multiplicación de matrices en lugar de dos millones de operaciones de conjunto en Python.

La **figura f17** muestra la distribución, con el eje vertical en logaritmo porque la primera barra es tres órdenes de magnitud mayor que las demás y en escala lineal aplastaría la cola, que es la única parte interesante. Solo 42 pares de 1.999.000 superan 0,5, el 0,002 por ciento. Esa es la figura que justifica que LSH exista: comparar todos los pares para encontrar esa fracción es desperdicio.




**f17.** Distribución de la similitud de Jaccard exacta

**Sección 3. MinHash**

**La propiedad que lo sostiene.** Para una permutación aleatoria del universo de shingles, la probabilidad de que dos conjuntos tengan el mismo elemento mínimo es exactamente su similitud de Jaccard. La intuición: mirando solo los elementos que están en la unión de los dos conjuntos, el mínimo es uniformemente alguno de ellos, y coincide si y solo si cae en la intersección, lo que ocurre con probabilidad igual a intersección sobre unión.

**Cómo se implementa sin permutar de verdad.** Permutar un universo de 137.412 shingles sería absurdo. En su lugar se usan funciones hash lineales de la forma a por x más b, módulo P, con P igual a 2 elevado a 31 menos 1, un primo de Mersenne. Cada par de coeficientes a y b sorteados define una permutación aproximada. La firma del conjunto es, para cada una de las n funciones, el mínimo de los valores hash de sus elementos.

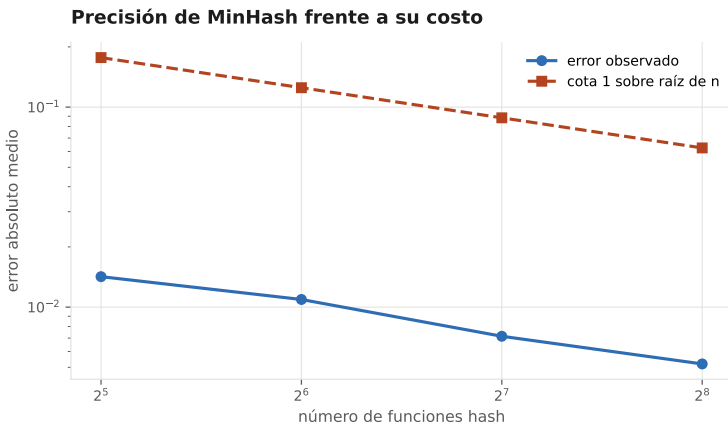
Dos detalles de implementación que conviene saber explicar.

1. **El hash base es blake2b y no el de Python.** La función hash incorporada de Python viene aleatorizado por proceso, lo que haría irreproducibles las firmas y los buckets entre ejecuciones y, peor, entre el driver y los workers de Spark.
2. **P es un primo de Mersenne de 31 bits** para que el producto a por x quepa en un entero de 64 bits sin desbordar, lo que permite vectorizar con numpy: una sola matriz de n por tamaño del conjunto y un mínimo por fila, en vez de iterar en Python.

**Cuánto error tiene.** La firma estima Jaccard como la fracción de posiciones en que las dos firmas coinciden. El error es del orden de uno sobre la raíz de n.

| Funciones hash | Error medio observado | Cota uno sobre raíz de n | Bytes por documento |
|---|---|---|---|
| 32 | 0,0142 | 0,1768 | 256 |
| 64 | 0,0109 | 0,1250 | 512 |
| 128 | 0,0072 | 0,0884 | 1.024 |
| 256 | 0,0052 | 0,0625 | 2.048 |

La **figura f18** muestra las dos curvas en escala logarítmica doble y tienen la misma pendiente, lo que confirma la ley. La consecuencia práctica es dura: cuadruplicar las funciones hash solo divide el error entre dos. Se eligen **128** porque su error, 0,0072, es un orden de magnitud menor que el ancho de la banda de decisión de LSH, así que afinar más la estimación por debajo de la resolución con la que luego se decide no compra nada.



**f18.** Error de MinHash según el número de funciones hash

## Sección 4. LSH por bandas, en Spark

**Cómo funciona.** La firma de 128 enteros se parte en b bandas de r filas. Cada banda se resume en un hash, y dos documentos son candidatos si coinciden en el hash de **al menos una** banda. Solo los candidatos se verifican con Jaccard exacta.

La probabilidad de que un par con similitud real s sea candidato es uno menos, uno menos s elevado a r, elevado a b. Esa es la **curva S**.

1. La probabilidad de coincidir en una banda concreta es s elevado a r, porque la banda tiene r posiciones y todas deben coincidir.
2. La probabilidad de no coincidir en ninguna de las b bandas es, uno menos s elevado a r, elevado a b.
3. El complemento es la fórmula.

**El umbral.** La aproximación estándar del punto de inflexión es uno sobre b, elevado a uno sobre r. Aquí hay un error frecuente que conviene no cometer: en ese umbral la probabilidad **no vale un medio**. Vale uno menos, uno menos uno sobre b, elevado a b, que tiende a uno menos uno sobre e, alrededor de **0,63**. El umbral marca dónde la curva cambia de concavidad, no dónde vale la mitad. Lo verificamos numéricamente antes de escribirlo.

**Detalle de implementación.** Con r igual a 3 salen 42 bandas y sobran dos posiciones de las 128, porque 3 no divide a 128. La firma se recorta a b por r antes de partirla, y cada banda lleva su propia semilla de hash para que bandas distintas con el mismo contenido no colisionen entre sí.

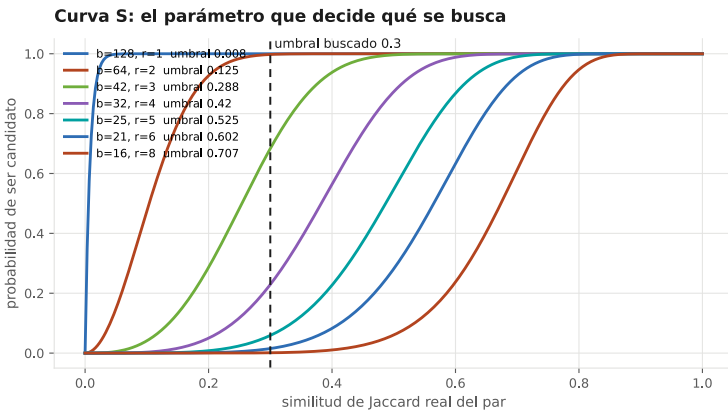
**Por qué RDD y no DataFrame.** El agrupamiento por bandas es un flatMap que emite un par por banda, seguido de un groupByKey y de las combinaciones dentro de cada bucket. Expresar eso en SQL sería forzado. Además se filtran los buckets con más de 400 elementos: un bucket enorme no informa, solo produce combinaciones cuadráticas de basura.

**LSH es un generador de candidatos, no un clasificador.** Esta frase decide cómo se mide todo lo que sigue. Su salida pasa después por una verificación exacta que elimina **todos** los falsos positivos, de modo que la precisión antes de verificar no es una medida de calidad sino de cuánto trabajo queda. Las dos cifras que importan son el recall, porque un par perdido aquí se pierde para siempre, y el número de candidatos, porque es el costo de la verificación.

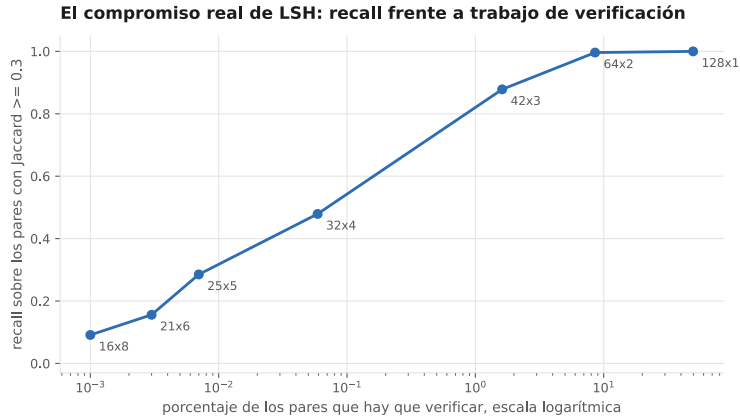
El barrido, con umbral buscado de 0,3 y 263 pares verdaderos:

| Configura-ción | Candida-tos | Falsos positivos | Falsos negativos | Recall | Pares a verificar |
|---|---|---|---|---|---|
| b=128, r=1 | 993.962 | 993.699 | 0 | 1,0000 | 49,72 % |
| b=64, r=2 | 171.259 | 170.997 | 1 | 0,9962 | 8,57 % |
| b=42, r=3 | 32.337 | 32.106 | 32 | 0,8783 | 1,62 % |
| b=32, r=4 | 1.177 | 1.051 | 137 | 0,4791 | 0,06 % |
| b=25, r=5 | 139 | 64 | 188 | 0,2852 | 0,007 % |
| b=21, r=6 | 59 | 18 | 222 | 0,1559 | 0,003 % |
| b=16, r=8 | 24 | 0 | 239 | 0,0913 | 0,001 % |

Las dos figuras que salen de aquí son la **f19**, la curva S de cada configuración, y la **f20**, recall contra costo de verificación.




**f19.** Curva S de LSH según bandas y filas




**f20.** Recall contra costo de verificación en el barrido de LSH

**Por qué no se usa F1.** Lo intentamos primero con F1 y seleccionaba una configuración con recall por debajo de la mitad, que para un generador de candidatos es inservible. F1 trata los dos errores como equivalentes y aquí no lo son: un falso positivo lo elimina la verificación posterior, así que es trabajo; un falso negativo se pierde para siempre. La configuración elegida es **b igual a 64 y r igual a 2**, que recupera el 99,6 por ciento de los pares verdaderos verificando solo el 8,6 por ciento de los pares.

## Sección 5. SimHash frente a MinHash

**Qué es SimHash.** Otro esquema de hashing sensible a la localidad, pero para similitud **coseno** en lugar de Jaccard. Se sortean tantos hiperplanos aleatorios como bits quiera tener la firma. Cada bit dice de qué lado del hiperplano cae el vector del documento. La probabilidad de que dos documentos coincidan en un bit es uno menos el ángulo

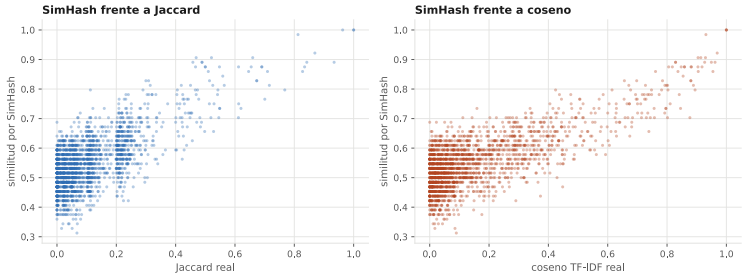
entre ellos dividido entre pi, de modo que la distancia de Hamming entre las dos firmas estima el ángulo y, con él, el coseno.

En la implementación, en lugar de sortear hiperplanos explícitos se usa el hash de cada término: cada bit del hash indica el signo con que ese término contribuye a esa coordenada, y se acumula sumando el peso del término con ese signo. Al final cada coordenada se reduce a un bit según su signo.

**Por qué los dos y no uno.** No son intercambiables. MinHash trata un conjunto **sin pesos**; SimHash respeta el peso de cada término. En este dataset la palabra adquisición aparece en casi todos los objetos y el nombre de un equipo médico en muy pocos. MinHash las trata igual; TF-IDF con SimHash no.

**TF-IDF.** Term Frequency por Inverse Document Frequency. Pesa cada término por lo frecuente que es en el documento y lo castiga por lo frecuente que es en la colección, de modo que lo que aparece en todas partes pesa poco.

El resultado, en la **figura f21** sobre 2.091 pares: SimHash correlaciona **0,734** con el coseno de TF-IDF y **0,694** con Jaccard. Cada esquema estima la métrica para la que fue diseñado.




**f21.** SimHash contra MinHash sobre los mismos pares

Dos cosas que conviene anticipar.

1. **Los pares están estratificados por similitud real, no elegidos al azar.** Con pares aleatorios casi todos tienen similitud cero y la correlación medida sería ruido, no una propiedad de SimHash.
2. **Ninguna correlación llega a uno, y el motivo es la resolución.** Con 64 bits la similitud estimada solo puede tomar 65 valores distintos, lo que produce las bandas horizontales visibles en los dos paneles. Subir a 128 bits afinaría la escala a costa del doble de memoria por firma.

### Sección 6. Contraste con Spark MLlib

Hay una **diferencia de contrato** que invalida la comparación ingenua. El método approxSimilarityJoin de MLlib no devuelve candidatos: genera los candidatos internamente y ya les aplica la verificación exacta, de modo que su salida son pares verificados. Comparar sus pares verificados contra los candidatos crudos de la

implementación propia daría una ventaja aparente enorme a MLlib que es solo de contabilidad.

Para que la comparación sea válida, a los candidatos propios se les aplica la misma verificación exacta.

| Configuración | Candidatos crudos | Pares verificados | Recall |
|---|---|---|---|
| propia, b=64 y r=2, verificada | 171.259 | 262 | 0,9962 |
| MLlib con 8 tablas hash | no expuesto | 252 | 0,9582 |
| MLlib con 16 tablas hash | no expuesto | 261 | 0,9924 |
| MLlib con 32 tablas hash | no expuesto | 261 | 0,9924 |

Con el mismo contrato las dos producen pares verificados correctos, y lo que queda es una diferencia de configuración y no de corrección. MLlib usa tablas hash independientes, que equivale a bandas de una sola fila, y eso le da un umbral efectivo muy bajo: atrapa casi todo a costa de generar muchos más candidatos internos, que no expone. La implementación propia permite elegir explícitamente ese punto con b y r, que es lo que el enunciado pide analizar.

**La verificación que importa.** Ningún par declarado por la implementación propia resulta falso tras la comprobación exacta: los 262 superan el umbral. Si el agrupamiento por bandas tuviera un error conceptual aparecerían pares que no lo superan, y no aparece ninguno. Y el recall queda al nivel de MLlib, 262 de 263 pares verdaderos frente a 261.

### Sección 7. Un caso concreto

Los cinco pares de mayor similitud. Los de la misma entidad son reconvocatorias del mismo objeto. El interesante es la Municipalidad Distrital de Tambo de Mora contra la Municipalidad Distrital de San Bartolo, dos municipalidades distintas con el mismo pliego del programa Vaso de Leche, una de 2024 y otra de 2023.

**Qué autoriza a concluir esto y qué no.** Un par con similitud alta dice que dos procesos describen casi lo mismo. Eso puede reflejar una plantilla compartida entre entidades, una necesidad genuinamente idéntica o un proceso fraccionado. Es una hipótesis que señala dónde mirar, no una conclusión sobre la conducta de nadie.

### Sección 8. A escala completa

Todo lo anterior corre sobre 2.000 procesos porque ahí se puede calcular la verdad de referencia. El método, en cambio, se aplica al conjunto entero en Spark: 166.439 procesos con objeto suficientemente largo, 13.850.887.141 pares posibles, 28.161.739 candidatos. Una reducción de **492 a 1**.

## 4. Alternativas para cada decisión, explicadas

### Para encontrar pares parecidos sin compararlos todos

1. **Fuerza bruta.** Quince mil millones de comparaciones. Es la cota contra la que se mide todo, y es lo que la

figura f17 demuestra que sería desperdicio: solo el 0,002 por ciento de los pares supera 0,5.

2. **Bloqueo, o vecindario ordenado.** Se ordenan los documentos por una clave, por ejemplo las primeras letras del objeto, y solo se comparan los que caen en la misma ventana. Es barato y es lo que se hacía antes de LSH. El problema es que es frágil: un error de digitación en la primera letra manda el documento a otro bloque y se pierde para siempre, y no hay forma de calcular la probabilidad de que eso pase. LSH tiene la misma idea pero con una **curva de probabilidad calculable**, que es lo que permite elegir el punto de operación a conciencia.
3. **Filtrado por prefijo, al estilo PPJoin.** Ordena los elementos del conjunto por frecuencia global y usa el hecho de que dos conjuntos con Jaccard alta deben compartir algo de su prefijo. Da resultados exactos sin falsos negativos y es más eficiente que la fuerza bruta, pero su costo depende mucho de la distribución y no se paraleliza tan limpiamente como el agrupamiento por bandas.
4. **Índice invertido con filtro de longitud.** Exacto y bueno cuando los conjuntos son pequeños. Aquí cada documento tiene 187 shingles y el universo es de 137.412, de modo que las listas de posteo serían enormes.
5. **Lo que se hizo.** LSH por bandas sobre firmas MinHash, porque es lo único de la lista que ofrece una curva S con parámetros explícitos y se expresa de forma natural como flatMap más groupByKey en Spark.

**Para resumir un conjunto en una firma**

1. **Permutaciones reales del universo.** Es la definición del algoritmo y es inviable: permutar 137.412 shingles por cada una de las 128 funciones.
2. **MinHash con hashes lineales.** Lo que se usó: a por x más b, módulo un primo de Mersenne. Cada par de coeficientes es una permutación aproximada.
3. **Hashing de una sola permutación con densificación.** Una sola función hash y el universo partido en n franjas, tomando el mínimo de cada franja. Cuesta una pasada en lugar de n, pero hay que tratar las franjas vacías con un esquema de densificación que complica la explicación. Con 128 hashes vectorizados en numpy no hacía falta.
4. **Bocetos de los k mínimos, o bottom-k.** Se queda con los k valores hash más pequeños del conjunto entero, en vez de el mínimo de k funciones. Estima mejor para el mismo tamaño de firma, pero no se parte en bandas de forma natural, que es justo lo que LSH necesita.
5. **Minwise hashing de b bits.** Guarda solo los b bits bajos de cada mínimo, de modo que la firma ocupa mucho menos a cambio de más colisiones. Es la optimización evidente si la memoria apretara; con 1 KiB por documento no aprieta.

**Para la representación del documento**

1. **Shingles de palabras.** Más interpretables y más baratos, pero frágiles ante errores de digitación,

abreviaturas y plurales, que abundan porque el objeto lo escribe cada entidad a mano.

2. **Shingles de caracteres.** Lo que se usó, con k igual a 9 fijado por criterio.
3. **El conjunto de códigos CUBSO.** Medido y descartado: 0,94 elementos por proceso y 270 procesos vacíos de 2.000.
4. **TF-IDF.** Sí se usa, pero para SimHash y para la Parte III, porque es una representación **con pesos** y Jaccard no los admite.

**Para la métrica de similitud**

1. **Jaccard.** Para conjuntos sin peso. Es la que estima MinHash.
2. **Coseno.** Para vectores con peso. Es la que estima SimHash, y la de la Parte III.
3. **Distancia de edición, o Levenshtein.** Es la métrica natural para errores de digitación, y se descarta por dos motivos: es cuadrática en la longitud de los textos, y no se conoce una familia de funciones sensibles a la localidad para ella, de modo que no admite el truco de LSH. Los shingles de caracteres son la forma práctica de capturar parte de esa tolerancia dentro de una métrica que sí la admite.
4. **Distancia de Hamming.** Es la que se usa **entre firmas SimHash**, no entre documentos.

**Para elegir bandas y filas**

1. **F1 sobre los candidatos crudos.** Fue el primer intento y seleccionaba una configuración con recall por debajo de la mitad. Trata los dos errores como equivalentes y aquí no lo son.
2. **Precisión.** Peor todavía: premia a la configuración que menos propone.
3. **Fijar el umbral teórico y despejar.** Se puede, y de hecho la fórmula está implementada, pero el umbral teórico es una aproximación y conviene comprobar con el barrido lo que realmente ocurre.
4. **Lo que se hizo.** Recall contra porcentaje de pares a verificar, eligiendo la configuración de menor costo entre las que superan el 95 por ciento de recall.

**Para el contraste con una biblioteca**

1. **Comparar los candidatos propios contra la salida de MLlib.** Inválido: son contratos distintos, porque MLlib ya verifica internamente.
2. **Lo que se hizo.** Aplicar a los candidatos propios la misma verificación exacta, y entonces comparar recall contra recall.

## 5. Preguntas probables y qué responder

**¿Cuál es el criterio para k y qué pasa si se elige otro?** El criterio es que el fondo de pares no relacionados quede por debajo de 0,02, y el primer k que lo cumple es 9. No

es crítico: con 7 y con 11 el barrido de LSH da resultados muy parecidos.

**¿El umbral de LSH es donde la probabilidad vale un medio?** No. La fórmula marca el punto de inflexión, donde la probabilidad vale uno menos uno sobre e, unos 0,63.

**¿Por qué 128 funciones hash y no más?** Porque el error con 128 ya es un orden de magnitud menor que el ancho de la banda de decisión de LSH; gastar más afina una estimación por debajo de la resolución con la que se decide.

**¿Por qué ninguna correlación de SimHash llega a uno?** Por resolución: con 64 bits solo hay 65 valores posibles de similitud estimada.

**¿Cómo saben que la implementación propia es correcta?** Porque todos sus pares verificados superan el umbral y porque coinciden con los de MLlib bajo el mismo contrato. Las dos comprobaciones hacen falta: la primera descarta falsos, la segunda descarta omisiones.

# Notebook 3. Búsqueda aproximada de vecinos: IVF y HNSW

Guía de estudio del notebook 3_ann_ivf_hnsw.ipynb. El porqué de cada elección está en docs/JUSTIFICACION.md, sección Parte III.

Qué significa cada campo, cada código CUBSO y cada etiqueta del dato está en docs/GLOSARIO.md.

## 1. Qué responde esta parte

Dado un proceso de contratación, cuáles son los diez más parecidos de todo el conjunto, y cuánto cuesta responder eso de forma exacta frente a aproximada.

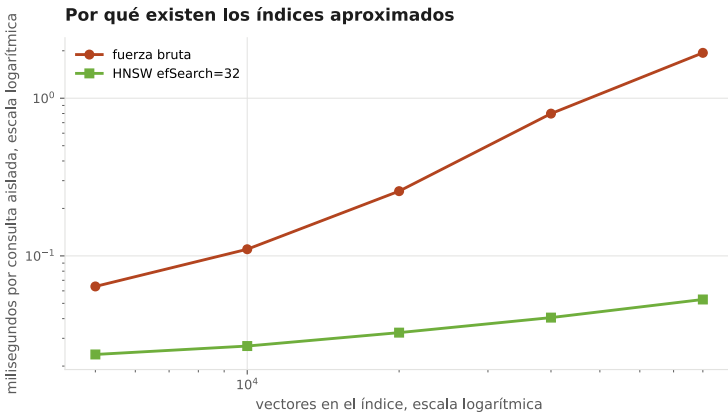
Es la misma pregunta de parecido de la Parte II, pero con otra representación y otra estructura. Allí el documento era un conjunto de shingles y la métrica Jaccard; aquí es un vector denso y la métrica el coseno. No es una cadena: son dos caminos distintos para la misma pregunta.

**Spark no se usa aquí, y hay que argumentarlo.** FAISS es una biblioteca de un solo nodo, el subconjunto cabe holgadamente en memoria, y toda la consulta es búsqueda en un índice y no agregación distribuida. Distribuir agregaría barajado y serialización sin ganar nada. Reconocer dónde Spark no aporta es parte de lo que evalúa la Parte VI.

## 2. Entradas y salidas

**Entrada.** El Parquet de la Parte I, filtrado a procesos con objeto de más de 60 caracteres y muestreado a 80.000. Ese tamaño está elegido para que la fuerza bruta duela y el índice tenga sentido, y para que el notebook corra en minutos.

**Salida.** Cinco artefactos y cinco figuras, de la f22 a la f26.




**f26.** Escalado del tiempo con el tamaño de la base

## 3. Recorrido sección por sección

### Sección 1. Dos representaciones vectoriales

Se construyen dos a propósito, para poder separar el efecto de la representación del efecto del índice.

1. **TF-IDF de palabras**, 22.037 términos. Es la de referencia.
2. **TF-IDF de 4-gramas de caracteres** con el analizador char_wb, 54.286 n-gramas. Es robusta frente a errores de digitación, que abundan porque el objeto lo escribe a mano cada entidad.

Las dos pasan por el mismo tratamiento: TF-IDF disperso, reducción a 128 dimensiones con **SVD truncada**, y normalización a norma uno.

**Por qué SVD.** FAISS trabaja sobre vectores densos y de dimensión fija. Un TF-IDF con 22.000 columnas sería inviable. La SVD truncada, que aplicada a TF-IDF se conoce como **Latent Semantic Analysis**, proyecta a un espacio de 128 dimensiones conservando la mayor parte de la varianza y, de paso, hace que términos que coocurren queden cerca.

**Por qué normalizar.** Con los vectores de norma uno, el **producto interno es exactamente el coseno**. Así se puede usar el índice de producto interno de FAISS y leer el resultado como similitud coseno sin convertir nada.

**Qué es sublinear_tf.** En lugar de la frecuencia bruta del término usa uno más su logaritmo, de modo que un término repetido diez veces no pesa diez veces más que uno que aparece una vez.

### Sección 2. Cómo se mide la latencia, y por qué el detalle decide la conclusión

Esta sección es la más importante del notebook y conviene saber contarla, porque corrige un error que

cometimos nosotros.

**El problema.** FAISS cambia de implementación según el tamaño del lote de consultas. Con un lote grande, el índice plano resuelve la búsqueda como un producto de matrices con BLAS y aprovecha los ocho núcleos. Con una sola consulta no tiene nada que amortizar.

**Los dos regímenes corresponden a usos reales distintos.**

1. **Por lote.** Un trabajo fuera de línea, por ejemplo deduplicar todo el catálogo de una vez.
2. **Consulta única.** Un servicio en línea que responde a una petición a la vez.

Se miden los dos y toda comparación usa el mismo régimen en ambos lados. Mezclarlos produce diferencias de un orden de magnitud que no son del índice sino del método de medición. La primera versión del notebook medía la fuerza bruta con lotes de mil consultas y el escalado con lotes de trescientas, lo que daba un falso multiplicador de 28.

**Por qué la consulta única se mide con un solo hilo de OpenMP.** Por dos razones. La primera es de realismo: un servicio en línea atiende cada petición en su hilo y no reparte una sola consulta entre los ocho núcleos. La segunda es de estabilidad: con ocho hilos el arranque del grupo de hilos domina el tiempo de una consulta individual, y la medición se vuelve sensible a la carga de la máquina hasta producir curvas no monótonas que no dicen nada del índice.

Además hay calentamiento: se descartan las veinte

primeras consultas y se toma la **mediana** de doscientas, no la media, porque la mediana no se mueve por una pausa del recolector de basura.

**La verdad de referencia** son los diez vecinos que devuelve el índice plano exacto, y la métrica de calidad es **recall en 10**: de los diez vecinos verdaderos, cuántos devuelve el índice aproximado.

### Sección 3. IVF, listas invertidas

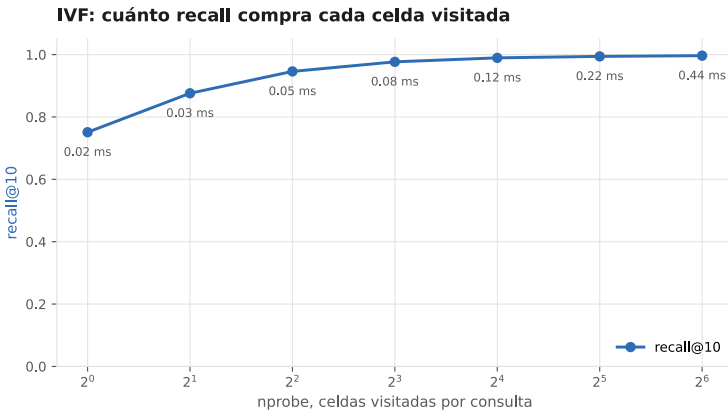
**Cómo funciona.** Un k-medias previo parte el espacio en nlist celdas de Voronoi y guarda cada vector en la lista de su centroide más cercano. Eso es el entrenamiento. En consulta, se calcula la distancia a los nlist centroides, se eligen las nprobe celdas más cercanas y se recorren solo esos vectores. El trabajo baja en proporción a nprobe sobre nlist.

**El parámetro de calidad es nprobe.** Visitar más celdas acerca al resultado exacto y cuesta más tiempo. El valor de nlist se fija con la regla habitual, la raíz del número de vectores, que aquí da 282.

**El error que comete.** Un vecino verdadero puede estar en una celda que no se visitó, porque la frontera de Voronoi no respeta la vecindad: dos puntos cercanos pueden caer en celdas distintas si están junto a la frontera.

La **figura f22** muestra que el recall sube muy rápido y luego se aplana. Con nprobe igual a 1 se visita el 0,35 por ciento de las celdas y ya se recupera el 75 por ciento de los vecinos verdaderos; con 16 se llega al 99 por ciento

en 0,122 milisegundos por consulta; con 64 se sube al 100 por ciento cuadruplicando el trabajo. La curva dice dónde dejar de pagar: más allá de la rodilla cada punto de recall cuesta desproporcionadamente.




**f22.** Recall y latencia de IVF según nprobe

## Sección 4. HNSW, grafo navegable en capas

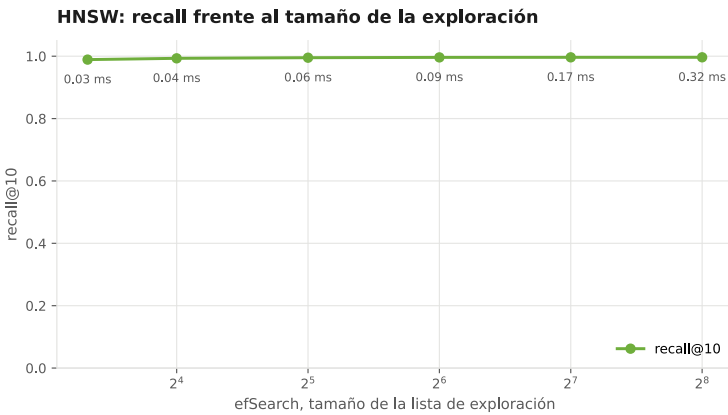
**Qué significa el nombre.** Hierarchical Navigable Small World. Es un grafo de vecindad con varias capas.

**Cómo funciona.** Cada vector se conecta con hasta M vecinos cercanos. Las capas superiores son dispersas, con pocos nodos y aristas largas, y sirven para acercarse rápido a la región correcta; las inferiores son densas y afinan. La consulta entra por la capa superior, avanza con una búsqueda voraz hacia el vecino que más reduce la distancia, y al no poder mejorar baja una capa y repite. En

la capa cero hace una búsqueda en haz con una lista de candidatos de tamaño efSearch.

**Los parámetros.** M es el número de aristas por nodo, aquí 32. efConstruction controla la calidad del grafo al construirlo, aquí 200. efSearch es el parámetro de calidad en consulta.

La **figura f23** muestra que HNSW llega muy alto con exploración mínima: con efSearch igual a 10 ya alcanza el 99 por ciento de recall en 0,028 milisegundos, y con 256 llega al 100 por ciento.



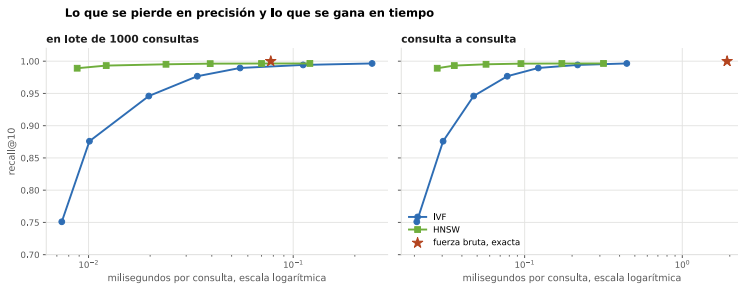

**f23.** Recall y latencia de HNSW según efSearch

**La contrapartida está fuera de la figura y hay que decirla.** Construir el grafo tomó 7,1 segundos frente a 0,4 de IVF, y el grafo guarda hasta 32 aristas por nodo además de los vectores: 61,4 MB frente a 41,1 de IVF y 41,0 de la fuerza bruta.

## Sección 5. Los dos juntos, frente a la fuerza bruta

La comparación honesta no es recall contra parámetro sino **recall contra latencia**, cada índice en su mejor configuración para cada presupuesto de tiempo.

La **figura f24** tiene dos paneles y cuentan historias opuestas, que es por lo que hacen falta los dos.




**f24.** Lo aproximado contra lo exacto, en lote y consulta a consulta

1. **En lote**, la fuerza bruta tarda 0,0777 milisegundos por consulta y el mejor HNSW sobre el 95 por ciento de recall tarda 0,0088. El índice aproximado aporta poco, porque el índice plano resuelve el lote como un producto de matrices con BLAS sobre ocho núcleos y eso es extremadamente eficiente.
2. **Consulta a consulta** se invierte: la fuerza bruta tarda 1,924 milisegundos, HNSW 0,028, es decir 69 veces más rápido, e IVF 0,078.

La moraleja: la pregunta de si un índice aproximado vale la pena no tiene respuesta sin decir antes si el uso es por

lotes o en línea.

**El escalado, figura f26.** Una sola medición no responde a partir de qué tamaño compensa, así que se mide sobre subconjuntos crecientes, en régimen de consulta única y con el mismo número de consultas en todos los puntos.

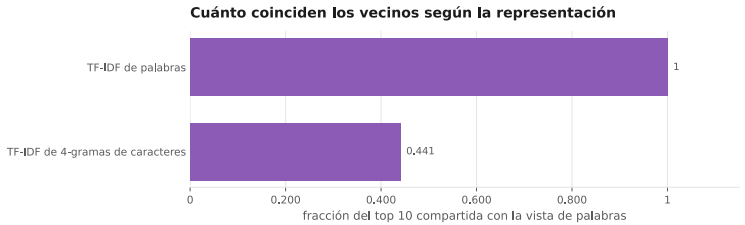
| Vectores | Fuerza bruta | HNSW | Factor |
|---|---|---|---|
| 5.000 | 0,064 ms | 0,024 ms | 2,7 |
| 10.000 | 0,110 ms | 0,027 ms | 4,1 |
| 20.000 | 0,257 ms | 0,033 ms | 7,9 |
| 40.000 | 0,799 ms | 0,041 ms | 19,7 |
| 80.000 | 1,940 ms | 0,053 ms | 36,7 |

Las dos rectas tienen pendientes distintas y esa diferencia es todo el argumento. Multiplicar la base por 16 multiplica el tiempo de la fuerza bruta por 30, que es crecimiento lineal, y el de HNSW por 2,2, que es el crecimiento logarítmico que promete la estructura.

**La conclusión incómoda que preferimos decir.** A nuestro volumen y en uso por lotes el índice aproximado no era necesario. Lo que lo justifica es el uso en línea y el escenario de volumen diez o cien veces mayor que plantea la Parte VI, y eso está medido aquí en lugar de supuesto.

**El efecto de la representación, figura f25.** Con el índice fijo y exacto en los dos casos, las dos representaciones comparten solo el 44 por ciento de los diez vecinos. Es un recordatorio incómodo: una parte grande de lo que se llama error de un índice aproximado es en realidad una decisión de representación tomada mucho antes. Los n-gramas de caracteres agrupan por forma de escritura;

las palabras agrupan por vocabulario compartido. Ninguna es la correcta en abstracto.




**f25.** Dos representaciones del mismo texto, mismo índice exacto

## Sección 6. Cuándo conviene cada uno

| Índice | Construcción | Memoria | Latencia al 95 por ciento de recall |
|---|---|---|---|
| Fuerza bruta | 0,0 s | 41,0 MB | 1,924 ms |
| IVF con 282 celdas | 0,4 s | 41,1 MB | 0,078 ms |
| HNSW con M igual a 32 | 7,1 s | 61,4 MB | 0,028 ms |

1. **IVF** cuando la memoria es el límite y el conjunto cambia seguido. Se construye rápido, ocupa poco por encima de los vectores y admite insertar sin reconstruir. Degrada de forma predecible.
2. **HNSW** cuando la latencia es el límite y el conjunto es estable. Da más recall por milisegundo, pero el grafo cuesta construirlo y ocupa más memoria, y borrar elementos es incómodo.

3. **Fuerza bruta** cuando el conjunto es pequeño o la consulta es rara. No tiene ningún parámetro que calibrar mal y es la verdad de referencia.

## 4. Alternativas para cada decisión, explicadas

### Para indexar vecinos

1. **Fuerza bruta, índice plano.** Exacta y sin parámetros. Se conserva siempre, porque es la verdad de referencia sin la cual el recall no se puede medir. Su costo crece de forma lineal con la base.
2. **Árboles k-dimensionales y árboles de bolas.** Parten el espacio con hiperplanos o esferas y podan ramas por desigualdad triangular. Funcionan muy bien hasta unas diez o veinte dimensiones y luego degradan hasta el recorrido completo, porque en dimensión alta casi todos los puntos acaban a distancias parecidas y la poda deja de podar. Es la maldición de la dimensionalidad, y con 128 dimensiones ya no aportan.
3. **IVF, listas invertidas.** Lo que se usó. k-medias y recorrido de las celdas más cercanas.
4. **HNSW, grafo navegable en capas.** Lo que se usó. Más recall por milisegundo, más memoria y más tiempo de construcción.
5. **IVFPQ, listas invertidas con cuantización por producto.** Parte el vector en subvectores y sustituye cada uno por el índice de su centroide en un pequeño libro de códigos, con lo que un vector de 128 flotantes pasa a ocupar unos pocos bytes. Es lo que se usa

cuando los vectores no caben en memoria. No se usa aquí por dos razones: 41 MB caben de sobra, y añadiría una **segunda fuente de error**, la de la compresión, que se confundiría con el error del índice, que es justo lo que esta parte quiere medir por separado.

6. **Árboles de proyección aleatoria, al estilo Annoy.** Construye un bosque de árboles que parten el espacio con hiperplanos aleatorios. Más simple que HNSW y en general peor en recall por milisegundo a este tamaño, pero con la ventaja de poder cargarse en memoria compartida desde disco.
7. **LSH para coseno.** Es exactamente SimHash, que ya está implementado en la Parte II. Para vectores densos y este rango de tamaños, IVF y HNSW lo dominan en recall por milisegundo; LSH brilla cuando se necesita una garantía probabilística explícita o un esquema distribuible, no cuando se quiere la menor latencia en un nodo.
8. **Fuerza bruta en tarjeta gráfica.** FAISS la soporta y a este volumen sería competitiva con cualquier índice. No se usa porque el entorno del proyecto es solo de procesador, y porque la pregunta del curso es sobre estructuras de índice y no sobre acelerar la fuerza bruta.

**Para la representación vectorial**

1. **TF-IDF disperso directo.** No sirve: FAISS necesita vectores densos de dimensión fija.
2. **TF-IDF más SVD truncada.** Lo que se usó. Aplicada a

TF-IDF esto se llama análisis semántico latente, y además de reducir la dimensión acerca términos que coocurren.

3. **Análisis de componentes principales.** Es casi lo mismo, pero centra los datos, cosa que sobre una matriz dispersa la vuelve densa y no cabría en memoria. La SVD truncada funciona sin centrar.
4. **Proyección aleatoria.** Más barata y con la garantía de Johnson y Lindenstrauss de que las distancias se conservan aproximadamente. Es una alternativa razonable y se descarta porque la SVD conserva mejor la estructura con la misma dimensión de llegada, y el costo de calcularla una vez es asumible.
5. **Incrustaciones de un modelo de lenguaje.** Darían similitud semántica de verdad y no de vocabulario. Se descartan a propósito: introducirían un componente que no se puede explicar línea por línea en una sustentación, y el curso trata de estructuras de índice y no de representaciones aprendidas.
6. **Normalizar a norma uno.** No es una alternativa sino un requisito del diseño: con norma uno el producto interno **es** el coseno, lo que permite usar el índice de producto interno y leer el resultado directamente.

**Para medir**

1. **Un solo régimen.** Es lo que se hace casi siempre y es lo que nos dio un falso multiplicador de 28. La conclusión cambia según el régimen, de modo que presentar uno solo sería engañoso.

2. **Media en lugar de mediana.** La media se mueve con una pausa del recolector de basura o con cualquier otro proceso de la máquina. Se usa la mediana de doscientas consultas, tras descartar veinte de calentamiento.
3. **Todos los hilos en la consulta única.** Se descarta porque el arranque del grupo de hilos domina el tiempo de una consulta individual y vuelve la medición sensible a la carga de la máquina, hasta producir curvas no monótonas.

**Para distribuir**

1. **Spark.** Descartado con argumento, no por comodidad: un nodo, 41 MB en memoria, y la operación es búsqueda en índice y no agregación. Distribuirlo añadiría barajado y serialización sin ganar nada. Si la base creciera cien veces, lo que se haría es **fragmentar** el índice por nodo y consultar todos los fragmentos en paralelo, combinando los diez mejores de cada uno, que es un patrón de dispersar y reunir y no un barajado.

## 5. Preguntas probables y qué responder

**¿Cuál de los dos números de latencia es el correcto?** Los dos, para preguntas distintas. El error está en comparar un lado medido en lote contra el otro medido consulta a consulta, que es lo que nos dio un falso multiplicador de 28 la primera vez.

**¿Dónde conviene dejar de subir nprobe?** En la rodilla de la curva. De 16 a 64 el recall sube un punto y el trabajo se cuadruplica.

**¿Entonces HNSW es mejor que IVF?** En latencia de consulta sí, con este volumen. Pero compra esa latencia con memoria y con tiempo de construcción, y con un índice que admite peor las inserciones masivas. Si la base cambia todo el tiempo o la memoria es escasa, IVF sigue siendo la elección.

**¿Cuál representación es la correcta?** Ninguna en abstracto. Depende de si el parecido que se busca es ortográfico o de vocabulario.

**¿Para este proyecto el índice aproximado era necesario?** A nuestro volumen y en uso por lotes, no, y preferimos decirlo. Lo justifica el uso en línea y el escenario de más volumen, y eso está medido.

# Notebook 4. Minería de flujos: muestreo, Bloom Filter, Count-Min Sketch y DGIM

Guía de estudio del notebook 4_flujos.ipynb. El porqué de cada elección está en docs/JUSTIFICACION.md, sección Parte IV.

Qué significa cada campo, cada código CUBSO y cada etiqueta del dato está en docs/GLOSARIO.md.

## 1. La restricción que da sentido a todo

Los 177.398 procesos se recorren en orden de convocatoria, uno a uno, y cada uno se ve **una sola vez**. No se puede volver atrás ni guardar el flujo completo. Esa es la diferencia entre un flujo y una tabla, y es lo que obliga a usar estructuras de resumen.

El flujo es simulado a partir del Parquet, ordenado por fecha, y va del 4 de enero de 2023 al 30 de diciembre de 2025. Hay que ser honesto en esto: no hay una fuente en tiempo real, se simula el régimen de una sola pasada para poder medir lo que el enunciado pide.

**Spark es opcional aquí y no se usa.** El recorrido es inherentemente secuencial y con estado: cada elemento modifica una estructura que el siguiente consulta. Distribuirlo exigiría combinar resúmenes parciales, lo que se puede hacer con Bloom y Count-Min porque son aditivos, pero no con DGIM sin reescribirlo. Spark se usa solo para ordenar y materializar el flujo.

**Qué hereda de la Parte I.** La pregunta abierta del suelo de procesos sin competencia. Sobre un flujo esa pregunta

se convierte en si el suelo está subiendo o bajando ahora mismo, y eso es un conteo sobre una ventana reciente, que es exactamente para lo que existe DGIM. Y el filtro de Bloom consulta el identificador del proveedor, que es la recurrencia que la figura f10 de la Parte I puso sobre la mesa.

## 2. Recorrido sección por sección

### Sección 1. Tres formas de muestrear

Tres técnicas que suelen confundirse. La diferencia no es de eficiencia sino de **qué preguntas sobreviven a la muestra**.

| Técnica | Eventos | Proveedo-res | Proveedores completos |
|---|---|---|---|
| Por evento, 10 por ciento | 17.849 | 11.065 | 46,9 % |
| Por clave, 10 por ciento de proveedores | 45.781 | 6.814 | 100 % |
| Reservoir de 17.739 elementos | 17.739 | 10.914 | 47,5 % |

1. **Muestreo por evento.** Cada evento entra con probabilidad fija e independiente. Conserva proporciones globales y destroza la integridad por entidad.
2. **Muestreo por clave.** Se sortea la **entidad**, no el evento, hasheando la clave y comparando contra el umbral. Si la entidad entra, entran todos sus eventos. Conserva integridad por entidad y pierde uniformidad sobre eventos.

3. **Reservoir sampling.** Mantiene una muestra uniforme de tamaño fijo k sin saber cuánto durará el flujo. Los primeros k elementos entran; el elemento número i, con i mayor que k, entra con probabilidad k sobre i y desaloja a uno al azar.

**Por qué el reservoir es uniforme, demostrado.** Esta es la parte bonita del algoritmo y conviene saber contarla, porque el enunciado del algoritmo parece magia hasta que se ve la inducción.

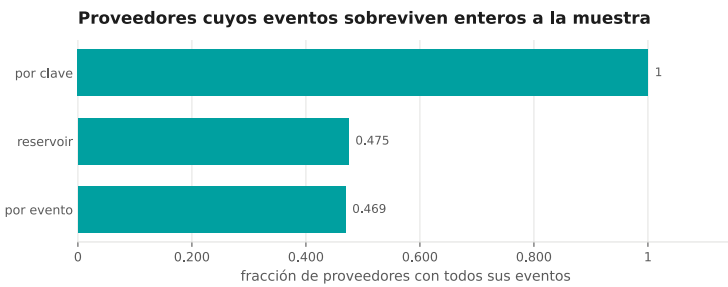
La afirmación es que, tras procesar i elementos, cada uno de los i está en la muestra con probabilidad k sobre i.

1. **Caso base.** Con i igual a k, los k primeros entraron todos, de modo que cada uno está con probabilidad k sobre k, que es uno.
2. **Paso inductivo.** Supongamos que vale tras i menos uno elementos, y llega el elemento i. Entra con probabilidad k sobre i, que es lo que queríamos para él. Para un elemento viejo x, que estaba con probabilidad k sobre i menos uno, hay que calcular la probabilidad de que sobreviva. Si el elemento i se rechaza, cosa que ocurre con probabilidad uno menos k sobre i, x sobrevive seguro. Si el elemento i se acepta, con probabilidad k sobre i, desaloja a una de las k posiciones elegida al azar, de modo que x sobrevive con probabilidad k menos uno, sobre k.
3. **La cuenta.** La probabilidad de que x siga dentro es k sobre i menos uno, multiplicado por la suma de los dos casos, que es uno menos k sobre i, más k sobre i por k menos uno sobre k. Esa suma vale uno menos k sobre i más k menos uno sobre i, es decir uno menos uno

sobre i, que es i menos uno sobre i. Y entonces k sobre i menos uno, por i menos uno sobre i, es **k sobre i**, que es lo que había que probar.

La consecuencia práctica es que el reservoir no necesita conocer el largo del flujo de antemano, que es exactamente lo que un muestreo del diez por ciento sí necesitaría para producir una muestra de tamaño acotado. Con un flujo sin final declarado, el muestreo por evento tiene memoria no acotada; el reservoir tiene memoria exactamente k.

**Por qué importa que una entidad quede completa.** Porque cambia qué preguntas admite la muestra. Sobre muestreo por evento, la pregunta cuántas adjudicaciones acumuló este proveedor da una respuesta **falsa**, no imprecisa: le faltan filas, y el conteo es sistemáticamente bajo. Con muestreo por clave la respuesta es exacta dentro de las entidades elegidas. Es la **figura f27**.

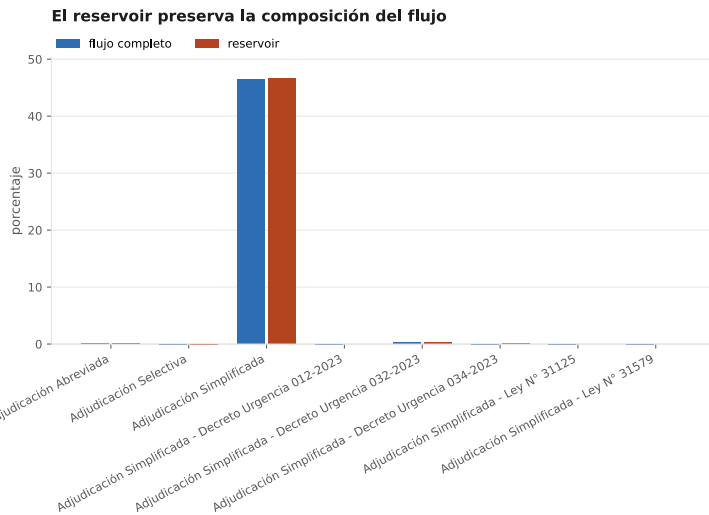



**f27.** Tres estrategias de muestreo sobre el mismo flujo

El reservoir se comporta como el muestreo por evento en

integridad, lo cual es esperado: su garantía es de uniformidad sobre eventos, no de integridad por entidad.

**La figura f28 verifica el reservoir.** El invariante es que, tras ver i elementos, cada uno está en la muestra con probabilidad k sobre i, y eso implica que la composición de la muestra debe reproducir la del flujo. La desviación máxima observada entre las dos distribuciones de métodos es de 0,06 puntos porcentuales.




**f28.** El reservoir reproduce la composición del flujo

Si preguntan para qué verificar algo ya demostrado: para distinguir un error de teoría de un error de implementación. La demostración garantiza el algoritmo, no nuestro código. Esta figura es la que detecta si el índice aleatorio está mal calculado, que es el fallo típico al escribir reservoir.

## Sección 2. Bloom Filter

**La pregunta.** Ha aparecido antes este proveedor, sin mantener el conjunto de todos los vistos.

**Cómo funciona.** Un arreglo de m bits, todos en cero, y k funciones hash. Insertar un elemento pone en uno las k posiciones que le corresponden. Consultar devuelve sí solo si las k posiciones están en uno.

**Por qué los falsos negativos son imposibles.** Por construcción, no por suerte: insertar solo enciende bits y nunca los apaga, así que si un elemento se insertó, sus k posiciones siguen encendidas. Esa asimetría es lo que lo hace útil como primer filtro: **un no es definitivo**.

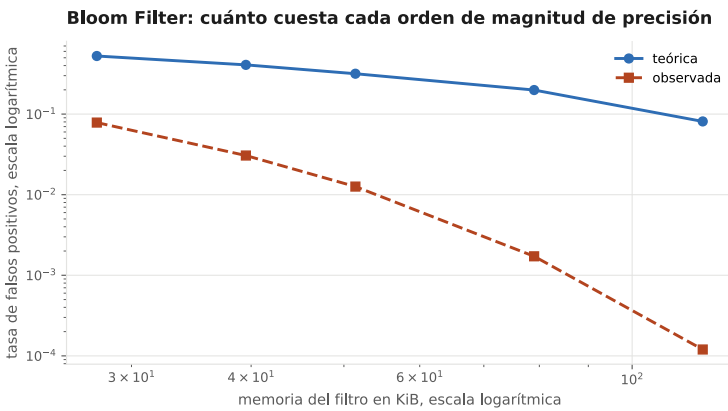
**Por qué puede haber falsos positivos.** Porque las k posiciones de un elemento ausente pueden haber sido encendidas por otros elementos.

**El dimensionamiento.** Para n elementos y tasa objetivo p, el tamaño óptimo es m igual a menos n por el logaritmo de p dividido entre el logaritmo de 2 al cuadrado, y el número óptimo de funciones hash es k igual a m sobre n por el logaritmo de 2. La tasa teórica es uno menos e elevado a menos k n sobre m, todo elevado a k.

**Un detalle de implementación que conviene saber.** No se calculan k hashes independientes, sino **doble hash de Kirsch y Mitzenmacher**: dos hashes h1 y h2, y la posición i es h1 más i por h2, módulo m. Se demuestra que tiene el mismo comportamiento asintótico que k funciones independientes y cuesta dos hashes en lugar de k.

| Tasa objetivo | Bits | Funciones hash | Memoria | Tasa observada | Falsos negativos |
|---|---|---|---|---|---|
| 0,200 | 226.148 | 2 | 27,6 KiB | 0,0785 | 0 |
| 0,100 | 323.544 | 3 | 39,5 KiB | 0,0306 | 0 |
| 0,050 | 420.940 | 4 | 51,4 KiB | 0,0126 | 0 |
| 0,010 | 647.088 | 7 | 79,0 KiB | 0,0017 | 0 |
| 0,001 | 970.631 | 10 | 118,5 KiB | 0,0001 | 0 |

La **figura f29** muestra que la curva observada sigue a la teórica, lo que valida la implementación. Lo que decide es la comparación con la alternativa exacta: guardar los 67.510 identificadores en un conjunto de Python costaría del orden de 1.582 KiB, y el filtro con 1 por ciento de error ocupa 79,0, unas 20 veces menos.




**f29.** Tasa de falsos positivos del filtro de Bloom

La tasa observada es menor que la teórica porque la teórica se evalúa con el filtro ya lleno, mientras que las consultas ocurren mientras se va llenando.

## Sección 3. Count-Min Sketch

**La pregunta.** Cuántas veces apareció cada código CUBSO, sin guardar un diccionario.

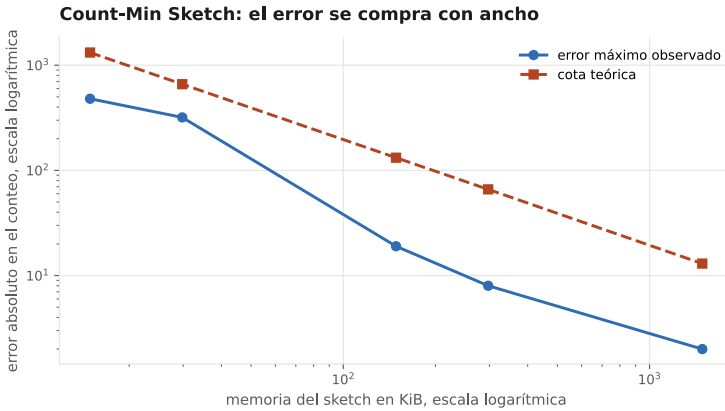
**Cómo funciona.** Una tabla de d filas por w columnas, inicialmente en cero, y d funciones hash, una por fila. Al llegar una clave se incrementa una celda en cada fila. Para estimar se toma el **mínimo** de las d celdas.

**Por qué el mínimo y por qué nunca subestima.** Cada celda acumula la clave más todas las que colisionan con ella, de modo que cada celda es una cota superior del conteo real. El mínimo de cotas superiores sigue siendo una cota superior, y es la más ajustada de las d.

**La garantía.** El error está acotado por epsilon por el total de eventos con probabilidad uno menos delta, donde w es e sobre epsilon y d es el logaritmo de uno sobre delta. El **ancho controla el error** y la **profundidad controla la probabilidad** de excederlo.

| Epsilon | Ancho | Profundi-dad | Memoria | Error medio | Error máximo | Cota teórica | Subestima-ciones |
|---|---|---|---|---|---|---|---|
| 0,0100 | 272 | 7 | 14,9 KiB | 97,24 | 480 | 1.320 | 0 |
| 0,0050 | 544 | 7 | 29,8 KiB | 30,70 | 319 | 660 | 0 |
| 0,0010 | 2.719 | 7 | 148,7 KiB | 0,57 | 19 | 132 | 0 |
| 0,0005 | 5.437 | 7 | 297,3 KiB | 0,04 | 8 | 66 | 0 |
| 0,0001 | 27.183 | 7 | 1.486,6 KiB | 0,00 | 2 | 13 | 0 |

La **figura f30** muestra que el error observado queda siempre por debajo de la cota y que nunca hay subestimaciones, que es exactamente lo que garantiza tomar el mínimo.




**f30.** Error del Count-Min Sketch según ancho y profundidad

**Aquí hay un resultado incómodo que se reporta igual.** El diccionario exacto de 4.664 códigos ocupa del orden de 273 KiB, y para bajar el error a dos apariciones el sketch necesita 1.487. A este volumen el Count-Min **no compensa**. Compensa cuando el universo de claves es mucho mayor que la memoria disponible, y nuestro universo de códigos CUBSO no lo es. Se incluye porque el curso lo pide y porque medir cuándo una estructura no conviene también es resultado.

## Sección 4. Por qué hacen falta los dos, y no uno de los dos

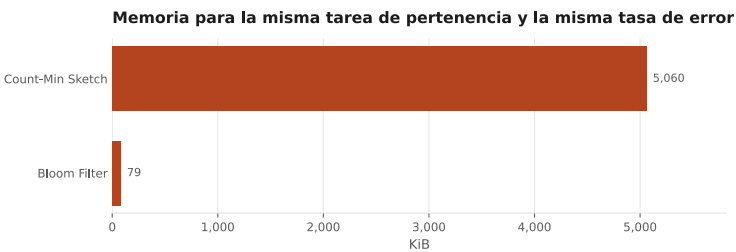
Es la pregunta natural: una frecuencia mayor que cero es una respuesta de pertenencia, entonces para qué un Bloom.

La respuesta se mide en lugar de afirmarse. El Count-Min se dimensiona resolviendo **su propia fórmula de**

**colisión**, no eligiendo parámetros a ojo: un elemento ausente se declara presente solo si las d filas tienen su celda ocupada, y con n claves en w columnas una celda está ocupada con probabilidad uno menos e elevado a menos n sobre w, así que hay que resolver esa expresión elevada a d igual a la tasa objetivo, y elegir la combinación de menor memoria.

| Estructura | Responde | Memoria | Tasa de falsos positivos |
|---|---|---|---|
| Bloom Filter | pertenencia | 79,0 KiB | 0,00172 |
| Count-Min Sketch | frecuencia | 5.059,6 KiB | 0,00167 |

A igual tasa de error el Count-Min necesita **64 veces más memoria**, y la razón es estructural: guarda un contador de ocho bytes donde al Bloom le basta un bit. Es la **figura f31**.




**f31.** Bloom contra Count-Min a igual tasa de error

Conviene contar que la primera versión de esta figura daba lo contrario, porque habíamos dimensionado el

Count-Min a ojo, y encontrarlo fue lo que nos obligó a derivar la fórmula.

### Sección 5. DGIM

**La pregunta.** Cuántos de los últimos N procesos tuvieron un solo postor. El flujo de bits es exactamente el indicador de postor único que la Parte I dejó abierto.

**Cómo funciona.** Se mantienen **cubos**, cada uno con una marca de tiempo y un tamaño que es potencia de dos, y a lo más r cubos de cada tamaño, con r igual a 2. Cuando llega un uno se crea un cubo de tamaño uno; si hay más de r cubos del mismo tamaño, los dos más antiguos se fusionan en uno del doble de tamaño, que hereda la marca de tiempo del más reciente de los dos. Los cubos que salen de la ventana se descartan.

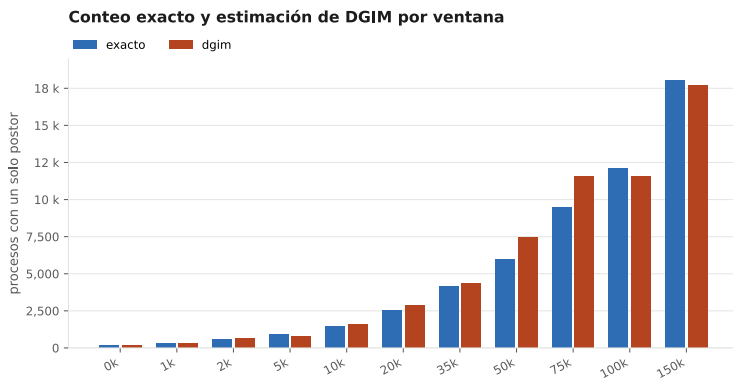
**Cómo estima.** Suma el tamaño de todos los cubos vivos **menos el más antiguo, del que cuenta solo la mitad**, porque de ese cubo no se sabe qué parte cae todavía dentro de la ventana.

**Por qué la memoria es logarítmica.** Hay a lo más r cubos de cada tamaño y los tamaños son potencias de dos hasta N, de modo que hay del orden del logaritmo de N cubos, cada uno con una marca de tiempo de del orden del logaritmo de N bits. De ahí el logaritmo al cuadrado.

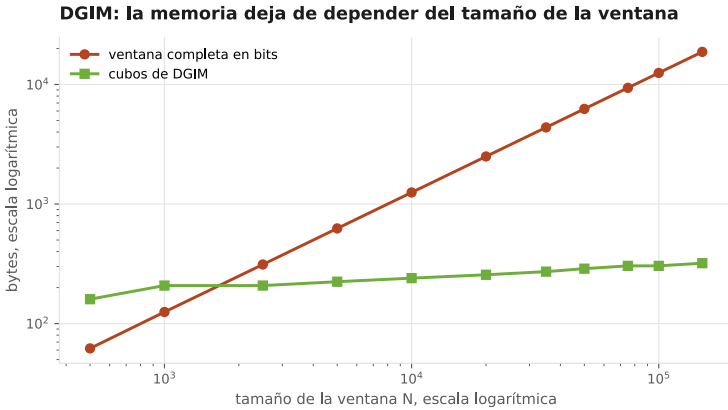
**La cota de error.** El único cubo del que se duda es el más antiguo, y su tamaño es a lo más la mitad del conteo, de modo que el error relativo está acotado por uno sobre r. Con r igual a 2 la cota es el 50 por ciento, y subiendo r baja a uno sobre r menos uno, a cambio de más memoria.

| Ventana | Exacto | DGIM | Error | Cubos | Memoria DGIM | Memoria exacta | Días cubiertos |
|---|---|---|---|---|---|---|---|
| 500 | 200 | 193 | 3,5 % | 10 | 160 B | 62 B | 25 |
| 5.000 | 879 | 793 | 9,8 % | 14 | 224 B | 625 B | 61 |
| 20.000 | 2.498 | 2.841 | 13,7 % | 16 | 256 B | 2.500 B | 138 |
| 50.000 | 5.959 | 7.449 | 25,0 % | 18 | 288 B | 6.250 B | 304 |
| 150.000 | 18.043 | 17.689 | 2,0 % | 20 | 320 B | 18.750 B | 902 |

La **figura f32** muestra que el peor error relativo observado es del 25 por ciento, muy por debajo de la cota teórica del 50. La **figura f33** muestra las dos curvas de memoria divergiendo: multiplicar la ventana por 300 multiplica la memoria exacta por lo mismo y los cubos pasan solo de 10 a 20.




**f32.** Error de DGIM sobre ventanas de distinto tamaño




**f33.** Memoria de DGIM contra la ventana exacta

**Un matiz que conviene decir.** Con ventanas pequeñas DGIM **pierde**: con N igual a 500 usa 160 bytes frente a 62 del conteo exacto, porque cada cubo guarda una marca de tiempo y eso cuesta más que unos pocos bits. El cruce aparece alrededor de los primeros miles de elementos. Afirmar que una estructura sublineal siempre gana es el error que esta figura evita.

## Sección 6. DGIM frente a reservoir

Se confunden porque ambos son resúmenes de un flujo con memoria acotada, pero responden a preguntas ortogonales. El reservoir da una muestra uniforme de **todo** el flujo, sin noción de recencia. DGIM responde solo sobre los últimos N elementos y olvida el resto por construcción.

| Método | Estimación | Memoria | Responde la pregunta de ventana | Error |
|---|---|---|---|---|
| Conteo exacto | 2.498 | 2.500 B | sí | 0 % |
| DGIM | 2.841 | 256 B | sí | 13,7 % |
| Reservoir filtrado | 2.820 | 32.000 B | no | 12,9 % |

El reservoir puede aproximar la pregunta de ventana solo descartando casi toda su muestra, y lo hace con más memoria y más error. A la inversa, DGIM no puede responder qué proporción de todo el histórico tuvo un solo postor, porque ya lo olvidó. Por eso se implementan los dos.

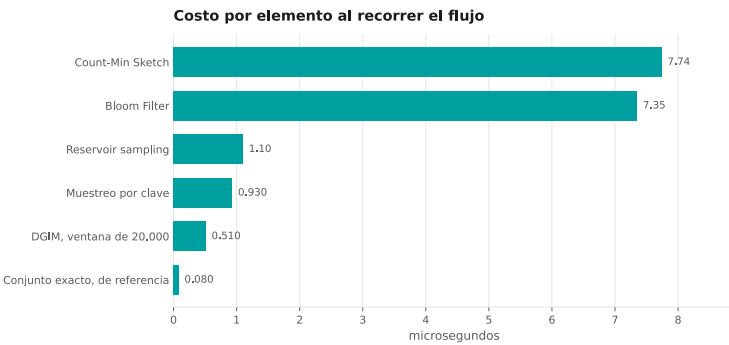
### Sección 7. Cuánto cuesta cada estructura en tiempo

El enunciado pide analizar precisión, memoria y tiempo. Las dos primeras ya están medidas; esta sección añade la tercera, que el discurso habitual sobre estas estructuras suele omitir.

| Estructura | Elementos | Microsegundos por elemento |
|---|---|---|
| Conjunto exacto de Python, referencia | 146.025 | 0,08 |
| DGIM con ventana de 20.000 | 177.398 | 0,51 |
| Muestreo por clave | 177.398 | 0,93 |
| Reservoir sampling | 177.398 | 1,10 |
| Bloom Filter | 146.025 | 7,35 |
| Count-Min Sketch | 132.091 | 7,74 |

El filtro de Bloom cuesta unas 92 veces más tiempo por elemento que el conjunto exacto de Python, porque calcula varias funciones hash donde el conjunto nativo calcula una y está implementado en C.

**La conclusión honesta, que es la que hay que decir:** estas estructuras **no ahorran tiempo**. Cambian memoria por tiempo y por un error acotado, y se eligen cuando la memoria es el recurso escaso. Presentarlas como una mejora en todos los ejes sería falso. Es la **figura f38**.




**f38.** Costo en tiempo de cada estructura

## Sección 8. Las cinco estructuras, lado a lado

La tabla es el entregable de la parte: no cuál es mejor, sino qué pregunta responde cada una y qué cuesta.

| Estructura | Qué pregunta responde | Memoria | Error | Por qué se usa |
|---|---|---|---|---|
| Muestreo por clave | métricas por entidad sobre una muestra | proporcio-nal a la fracción de claves | ninguno dentro de las claves elegidas | la única que conserva entidades completas |
| Reservoir sampling | proporcio-nes sobre todo el flujo | k elemen-tos, fija y conocida | 0,06 puntos de desviación | no necesita conocer el largo del flujo |
| Bloom Filter | apareció antes este proveedor | 79,0 KiB para 67.510 claves | 0,17 por ciento de falsos positivos, cero falsos negativos | un bit por posición, imbatible en pertenencia |
| Count-Min Sketch | cuántas veces apareció cada ítem | 149 KiB | nunca subestima, máximo 19 apariciones | única que da frecuencias sin diccionario |
| DGIM | cuántos en los últimos N elementos | 20 cubos para N igual a 150.000 | 25 por ciento observado, cota teórica 50 | única con noción de recencia |

## 3. Por qué cada estructura y no la otra

Las preguntas del tipo si usan DGIM, por qué también reservoir, o por qué no solo uno de los dos, son las más probables. La respuesta corta es que cada una responde una pregunta que la otra no puede responder. La respuesta larga, pareja por pareja:

**DGIM frente a reservoir.** DGIM responde cuántos unos hay en los últimos N elementos. El reservoir responde qué proporción tiene el flujo entero. El reservoir no tiene

noción de recencia: un evento de 2023 tiene la misma probabilidad de estar en la muestra que uno de 2025. Si se le fuerza la pregunta de ventana filtrando su muestra por posición, quedan 234 elementos de 2.000 y el resultado cuesta 32.000 bytes con 12,9 por ciento de error, frente a 256 bytes y 13,7 por ciento de DGIM. En sentido contrario, DGIM no puede responder qué proporción de todo el histórico tuvo un solo postor, porque el histórico ya lo olvidó. La tabla de la sección 6 del recorrido es esa medición.

**DGIM frente al muestreo por clave.** Ni siquiera compiten. El muestreo por clave decide qué entidades entran y conserva todos sus eventos, de modo que su eje es la entidad; el de DGIM es el tiempo. Si lo que se quiere es cuántos procesos sin competencia hubo en los últimos tres meses, el muestreo por clave da una respuesta sesgada por construcción, porque la respuesta depende de qué entidades salieron sorteadas y no del período.

**Reservoir frente al muestreo por clave.** Esta es la pareja que de verdad se confunde, porque las dos producen una muestra. La diferencia está medida en la figura f27: con muestreo por evento o reservoir, solo el 47 por ciento de los proveedores de la muestra conserva todos sus eventos, y con muestreo por clave el 100 por ciento. Entonces la pregunta cuántas adjudicaciones acumuló este proveedor da una respuesta falsa sobre el reservoir, no imprecisa, porque le faltan filas. Y a la inversa, el muestreo por clave no es uniforme sobre eventos: una entidad enorme entra entera o no entra, de modo que la varianza de cualquier proporción estimada es mucho mayor. Hacen falta los dos.

**Bloom frente a Count-Min.** Una frecuencia mayor que

cero es una respuesta de pertenencia, así que la pregunta de por qué los dos es legítima. La sección 4 del recorrido la responde con una medición, no con una opinión: a igual tasa de error, el Count-Min necesita 64 veces más memoria para responder pertenencia, por una diferencia estructural de ocho bytes por celda contra un bit.

**Cualquiera de ellas frente al conteo exacto.** Es la comparación que falta en casi toda exposición de este tema y que aquí se hace siempre. El Bloom gana claramente, 79 KiB contra 1.582. DGIM gana a partir de algunos miles de elementos y pierde por debajo. El Count-Min **pierde** a este volumen, 1.487 KiB contra 273, y se reporta así.

## 4. Técnicas del temario que no se implementaron, y por qué

Conviene saber que existen y poder decir en una frase qué hacen y por qué no hacían falta.

1. **HyperLogLog.** Responde cuántos elementos distintos hay en el flujo, con memoria del orden del logaritmo del logaritmo de la cardinalidad. Es la cuarta pregunta clásica sobre un flujo, junto a pertenencia, frecuencia y recencia. Sí se usa en el perfilado de la Parte I, con approx_count_distinct, pero no aquí: la cardinalidad exacta de proveedores, 67.510, cabe de sobra en memoria y no habría nada que ahorrar. Su antecesor es Flajolet y Martin, que estima la cardinalidad a partir del máximo número de ceros iniciales en los hashes; HyperLogLog lo mejora promediando muchos registros con media armónica y corrigiendo el sesgo.

2. **Misra y Gries, y Space-Saving.** Encuentran los elementos pesados, es decir los que superan una fracción del flujo, con memoria proporcional a uno sobre esa fracción. Son la alternativa al Count-Min cuando la pregunta no es cuántas veces apareció una clave cualquiera sino cuáles son las más frecuentes. No se usan porque nuestra pregunta es la primera: el conteo de una clave concreta, consultable después.
3. **Lossy Counting y Sticky Sampling, de Manku y Motwani.** Conteo aproximado de elementos frecuentes sobre flujos con garantía de error epsilon y sin falsos negativos. Mismo motivo que el anterior.
4. **Histogramas exponenciales, de Datar y otros.** Son la generalización de DGIM para sumar **valores** en una ventana y no solo contar unos. Es la extensión natural si en lugar de cuántos procesos sin competencia quisiéramos cuántos soles movieron esos procesos en los últimos N. No se implementó porque nuestra pregunta es un conteo sobre un indicador binario, que es exactamente el caso que DGIM cubre.
5. **Filtro de Bloom con contadores, y filtro cuckoo.** Variantes que permiten borrar elementos, cosa que el Bloom clásico no puede porque apagar bits introduciría falsos negativos. El cuckoo además ocupa menos a tasas de error bajas. No hacen falta: nuestro conjunto de proveedores vistos solo crece.
6. **Filtro de Bloom escalable.** Encadena filtros de tamaño creciente cuando no se conoce de antemano cuántos elementos habrá. Aquí sí se conoce, porque la cardinalidad se mide antes.
7. **Boceto de Alon, Matias y Szegedy.** Estima

momentos de frecuencia, por ejemplo la suma de los cuadrados, que mide lo desigual que es la distribución. Sería otra forma de medir concentración, pero la Parte I ya la mide de forma exacta con Gini porque la tabla cabe.

8. **Ventana deslizante exacta con una cola.** Es la alternativa exacta contra la que se compara DGIM. Para N igual a 150.000 cuesta 18.750 bytes frente a 320.
9. **Subir r en DGIM.** La cota de error es uno sobre r menos uno, de modo que con tres cubos por tamaño bajaría del 50 al 33 por ciento. No se hace porque el error observado ya está muy por debajo de la cota, pero es la palanca disponible y conviene saberlo.
10. **Spark.** Descartado con argumento: el recorrido es secuencial y con estado. Bloom y Count-Min sí son aditivos, de modo que esos dos se podrían distribuir combinando resúmenes parciales con un or lógico y una suma respectivamente; DGIM no, sin reescribirlo. Decir esa diferencia es parte de entender las estructuras.

## 5. Preguntas probables y qué responder

**¿Por qué no hay falsos negativos en el Bloom?** Por construcción: insertar solo enciende bits y nunca los apaga.

**¿En este caso el sketch realmente ahorró memoria?** No, y lo reportamos así. El diccionario exacto ocupa 273 KiB y el sketch necesita 1.487 para bajar el error a dos apariciones.

**¿De dónde sale el factor 64 entre Bloom y Count-Min?** De una diferencia estructural: ocho bytes por celda frente a un bit. No se sustituyen, responden preguntas distintas.

**¿Un error del 25 por ciento en DGIM no es demasiado?** Depende de la pregunta. Para decidir si una entidad entró en una racha inusual, un conteo con error acotado llega antes y basta. Para una cifra que se publica, no sirve. Y la cota se ajusta subiendo r.

**¿Entonces estas estructuras no sirven?** Sirven cuando la memoria es el recurso escaso, que es la situación de un flujo que no cabe. No ahorran tiempo: cambian memoria por tiempo y por error acotado.

**¿Por qué el flujo es simulado?** Porque no hay una fuente en tiempo real del OCDS peruano. Se ordena la tabla por fecha de convocatoria y se recorre una sola vez, que es el régimen que importa para medir estas estructuras.

# Notebook 5. Conjuntos frecuentes y reglas de asociación

Guía de estudio del notebook 5_reglas_asociacion.ipynb. El porqué de cada elección está en docs/JUSTIFICACION.md, sección Parte V.

## 1. Qué responde esta parte

La Parte I dejó un suelo de procesos sin competencia efectiva que atraviesa todos los tramos de monto. Falta saber si se reparte al azar o se concentra en combinaciones específicas de rubro, método y nivel de gobierno. Eso es, exactamente, una pregunta de patrones frecuentes.

**Y la decisión que determina todo el resultado es qué se considera una transacción.** El enunciado pide justificarla; aquí se justifica con una medición y no con una intuición, porque la definición obvia resultó no servir.

## 2. Qué es una entidad y por qué una contratación tiene canastas

**Entidad.** Es el comprador estatal, lo que en la tabla es el campo comprador_id. No es una empresa: en contratación pública el Estado es el que compra y la empresa está al otro lado del mostrador, como postor o como proveedor. Entidad es una municipalidad distrital, un gobierno regional, un hospital, una UGEL, un ministerio o un programa social. Hay 2.968 compradores

distintos en los tres años.

Esto importa porque **el sujeto de nuestras reglas es la entidad compradora y no el proveedor**. Agrupar por proveedor y mes era posible, y respondería qué le compran juntos a un mismo proveedor, pero la pregunta que hereda esta parte es sobre el comportamiento del comprador.

**La correspondencia con el supermercado**, que es la analogía con la que se enseña el tema:

| En el supermercado | En las contrataciones del Estado peruano |
|---|---|
| El cliente | La entidad pública compradora |
| El ticket de compra | Todo lo que esa entidad contrató en un mes |
| El producto del ticket | El código CUBSO del objeto contratado |

El ítem no es un producto de abarrotes: es un código del **CUBSO**, el Catálogo Único de Bienes, Servicios y Obras, así que el universo incluye insumos, servicios y obras. Por eso entre los ítems aparecen cosas como servicio de alquiler de excavadora hidráulica o servicio de seguridad y vigilancia. La estructura del código, dígito por dígito, está en docs/GLOSARIO.md.

**Por qué las reglas que salen tienen sentido.** No son el ejemplo del libro, son el resultado medido, y cada grupo tiene una causa institucional identificable.

1. Conserva de sardina o anchoveta, aceite vegetal comestible, arroz pilado superior, leche evaporada, hojuela de avena con quinua y kiwicha. Es la canasta de los **programas sociales de alimentación**: Vaso de

Leche, comedores populares y Qali Warma. La municipalidad que compra aceite en un mes compra también conserva y arroz porque está armando la misma ración.

2. Alquiler de camión volquete, cemento Portland, alquiler de excavadora hidráulica. Es la canasta de **obra pública**: maquinaria y material de la misma obra.
3. Servicio de limpieza de áreas administrativas junto con servicio de seguridad y vigilancia, con lift 8,8. Este es el mejor ejemplo para defender el método, porque demuestra que no está encontrando comida sino **paquetes de abastecimiento institucional**: la entidad renueva los dos contratos en el mismo mes.

La frase corta, si preguntan qué hace una canasta de supermercado en un dataset de contrataciones: **la entidad es el cliente, el mes es el ticket y el código CUBSO es el producto.**

## 3. El vocabulario mínimo

1. **Transacción o canasta.** El conjunto de ítems que ocurren juntos. En el supermercado, lo que lleva un cliente en un ticket.
2. **Conjunto frecuente.** Un conjunto de ítems cuyo soporte supera un umbral.
3. **Soporte.** La fracción de transacciones que contienen el conjunto. Es una probabilidad conjunta.
4. **Confianza** de la regla A implica B. El soporte del conjunto unión dividido entre el soporte de A. Es la

probabilidad condicional de B dado A.

5. **Lift.** La confianza dividida entre el soporte de B. Mide cuántas veces más aparece B junto a A de lo que cabría esperar si fueran independientes. Lift igual a uno es independencia.
6. **Antimonotonía.** Si un conjunto no es frecuente, ningún superconjunto suyo puede serlo. Es la propiedad que hace viable toda la minería de patrones.

## 4. Recorrido sección por sección

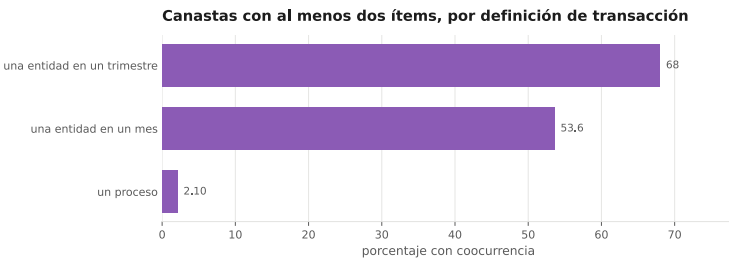
### Sección 1. Qué es una transacción aquí

La definición natural es que cada proceso es una canasta de códigos CUBSO. Se mide antes de usarla, junto con dos alternativas que agrupan por entidad y período.

| Definición | Canastas | Tamaño medio | Tamaño mediano | Con dos o más ítems |
|---|---|---|---|---|
| Un proceso | 177.398 | 0,74 | 1 | 2,1 % |
| Una entidad en un mes | 37.114 | 2,78 | 2 | 53,6 % |
| Una entidad en un trimestre | 21.098 | 4,49 | 2 | 68,0 % |

Esta es la **figura f34** y decide la parte entera. La definición natural **no sirve**: solo el 2,1 por ciento de los procesos tiene dos o más ítems, con tamaño medio de canasta por debajo de uno. Un conjunto de un solo elemento no contiene coocurrencia, de modo que A-Priori y FP-Growth sobre esa definición devolverían únicamente

conjuntos de tamaño uno, es decir un ranking de frecuencias disfrazado de minería de patrones.




**f34.** La canasta natural contra la canasta por entidad y mes

**El motivo, que hay que saber explicar.** El catálogo CUBSO identifica el objeto del proceso, y la mayoría de los procesos peruanos tienen un objeto único. La analogía del supermercado no se sostiene a nivel de acto administrativo: el acto administrativo no es el ticket, es el producto.

**Por qué mes y no trimestre.** El trimestre da más coocurrencia, 68 por ciento, pero se adopta el mes por dos razones. Conserva más transacciones, y un mismo umbral de soporte relativo es más exigente cuando hay más transacciones. Y es interpretable: la regla dice que la entidad que compra A compra también B dentro del mismo mes, lo que describe un patrón de abastecimiento. A escala trimestral la coocurrencia empieza a capturar simplemente que la entidad compra mucho.

Si preguntan si cambiar la definición hasta que salga algo no es hacer trampa: sería trampa si la definición nueva no tuviera sentido, y lo tiene. Además la figura documenta

**las tres** definiciones, incluida la que no funcionó, en lugar de presentar solo la que salió bien.

Las canastas unitarias se descartan después del agrupamiento porque no aportan coocurrencia.

## Sección 2. A-Priori, implementación propia

### Cómo funciona, por niveles.

1. Cuenta el soporte de cada ítem individual y se queda con los frecuentes.
2. Genera candidatos de tamaño dos uniendo pares de frecuentes de tamaño uno, cuenta su soporte con una pasada completa por las transacciones y se queda con los frecuentes.
3. Repite subiendo de nivel hasta que no quedan candidatos.

**Dónde entra la antimonotonía.** En la generación de candidatos. Un candidato de tamaño k solo se considera si **todos** sus subconjuntos de tamaño k menos uno son frecuentes. Esa poda es lo que evita el crecimiento combinatorio completo.

**Cuál es su costo.** Una pasada completa por las transacciones en cada nivel, y el conteo de todos los candidatos contra cada transacción. Cuando el umbral baja, el número de candidatos explota.

Se corre sobre una muestra de 8.000 transacciones, porque A-Priori no escala al conjunto completo, y eso también es un resultado que se reporta.

## Sección 3. FP-Growth con Spark MLlib

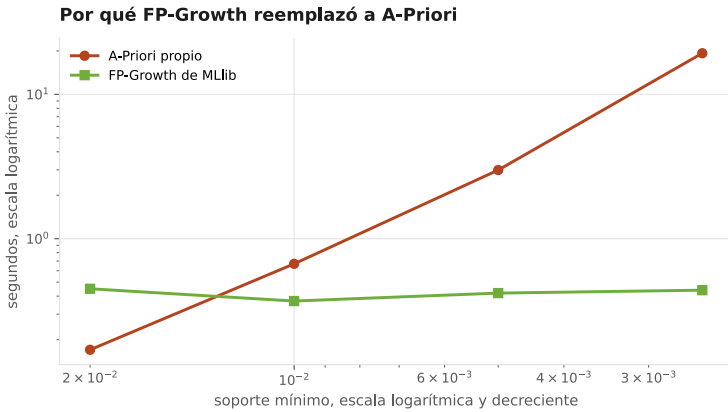
**Cómo funciona.** Dos pasadas por el dato y ningún candidato.

1. Primera pasada: cuenta la frecuencia de cada ítem y descarta los no frecuentes.
2. Segunda pasada: inserta cada transacción, con sus ítems ordenados por frecuencia descendente, en un **árbol de prefijos** llamado FP-tree. Las transacciones que comparten prefijo comparten camino, de modo que el árbol comprime el dataset.
3. Minería: para cada ítem, de menos a más frecuente, construye su **base de patrones condicional**, que son los caminos del árbol que terminan en ese ítem, y sobre ella construye un FP-tree condicional del que extrae recursivamente los conjuntos frecuentes.

**Por qué es más rápido.** No genera ni cuenta candidatos. El trabajo depende del tamaño del árbol comprimido y no del número de combinaciones posibles.

**La verificación.** Los dos algoritmos deben devolver exactamente los mismos conjuntos frecuentes sobre la misma muestra, y lo hacen: cero conjuntos exclusivos de uno u otro, y diferencia máxima de soporte nula. Esa comprobación es lo que permite confiar en FP-Growth para el conjunto completo y deja A-Priori como implementación de referencia.

**El costo según el umbral**, que es la **figura f35**:




**f35.** A-Priori contra FP-Growth al bajar el soporte

| Soporte mínimo | Conjuntos frecuentes | A-Priori | FP-Growth |
|---|---|---|---|
| 0,0200 | 34 | 0,17 s | 0,45 s |
| 0,0100 | 80 | 0,67 s | 0,37 s |
| 0,0050 | 202 | 2,99 s | 0,42 s |
| 0,0025 | 554 | 19,27 s | 0,44 s |

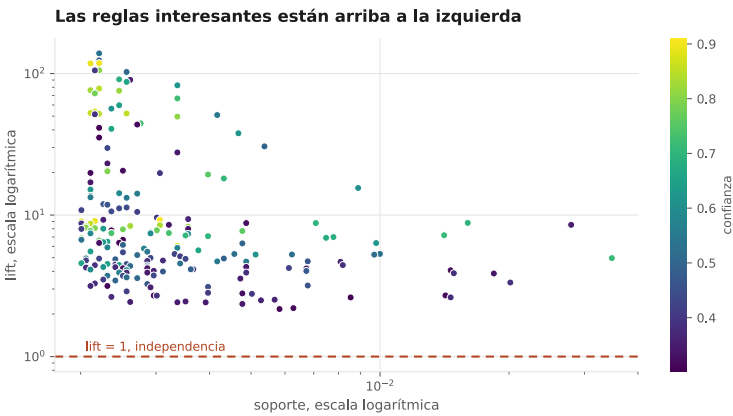
Con soporte alto los dos cuestan parecido porque hay pocos conjuntos frecuentes y A-Priori poda casi todo en el primer nivel. Al bajar el umbral de 0,02 a 0,0025, A-Priori pasa de 0,17 a 19,27 segundos y FP-Growth se mantiene plano. La diferencia es estructural y es exactamente la razón por la que el segundo desplazó al primero.

Si preguntan por qué implementar A-Priori si FP-Growth gana: porque A-Priori es donde se ve la antimonotonía, y porque sin medir la explosión de candidatos decir que FP-Growth es mejor sería repetir lo que dice el libro.

## Sección 4. Reglas sobre el conjunto completo

Sobre las 37.114 canastas de entidad y mes, con soporte mínimo 0,002 y confianza mínima 0,3.

La **figura f36** es una nube de soporte contra lift, coloreada por confianza, con los dos ejes en logaritmo y la línea de lift igual a uno marcada. Muestra el compromiso central de la minería de reglas: las reglas de soporte alto tienen lift cercano a uno, porque describen ítems que coinciden por ser ambos frecuentes y no porque uno prediga al otro; las de lift alto viven en la zona de soporte bajo, donde el patrón es fuerte pero afecta a pocas transacciones. El máximo de lift es 139.




**f36.** Soporte, confianza y lift de las reglas obtenidas

**Por qué el criterio de selección es el lift y no la confianza.** Porque la confianza no descuenta la frecuencia del consecuente. Una regla que concluye en

un ítem presente en la mitad de las canastas tiene confianza alta por construcción y no informa nada. El lift sí lo descuenta, porque compara contra la independencia.

Las diez reglas seleccionadas exigen soporte de al menos 0,004 y lift de al menos 3:

| Si la entidad compra | También compra | Soporte | Confianza | Lift |
|---|---|---|---|---|
| Conserva de sardina o anchoveta | Aceite vegetal comestible | 0,0042 | 0,386 | 50,9 |
| Aceite vegetal comestible | Conserva de sardina o anchoveta | 0,0042 | 0,550 | 50,9 |
| Aceite vegetal comestible | Arroz pilado superior | 0,0047 | 0,616 | 37,8 |
| Arroz pilado superior | Conserva de sardina o anchoveta | 0,0054 | 0,330 | 30,6 |
| Alquiler de camión volquete más cemento | Alquiler de excavadora hidráulica | 0,0043 | 0,705 | 18,1 |

**Cómo interpretarlas.** Las primeras son la canasta de los programas sociales de alimentación, sobre todo Vaso de Leche y los comedores: la entidad que compra aceite en un mes compra también conserva y arroz. Las últimas son la canasta de obra pública: maquinaria y materiales que se contratan juntos. Ninguna es sorprendente para quien conozca el sector, y eso es buena señal: el método recupera estructura real antes de buscar lo inesperado.

**Sección 5. Canasta enriquecida, donde se responde la pregunta de la Parte I**

Hasta aquí la canasta tiene solo códigos de catálogo, de modo que las reglas solo pueden hablar de qué se compra junto. Pero la pregunta que dejó abierta la Parte I es otra, y **tener un solo postor no es un producto del catálogo, es un atributo del proceso**.

La técnica, que conviene saber nombrar, es **convertir atributos categóricos en ítems** para que el mismo algoritmo de patrones frecuentes descubra asociaciones entre atributos y no solo entre productos. A cada canasta se le agregan los atributos del proceso como ítems, con un prefijo que nombra el campo. Una canasta enriquecida se ve así:

**{ RUBRO=5013, METODO=Contratación Directa, NIVEL=Gobierno nacional, TRAMO=50 mil a 500 mil, POSTOR=unico }**

Cuatro detalles de implementación que hay que saber justificar.

1. **El prefijo no es decoración.** Sin él un valor de un campo podría colisionar con el de otro, y la regla dejaría de leerse sola. Con él, el antecedente se lee en castellano.
2. **Aquí la canasta vuelve a ser el proceso**, no la entidad y el mes, porque los atributos pertenecen al proceso. Y ahora sí funciona, porque por construcción cada canasta tiene cinco ítems o más, así que hay coocurrencia de sobra. Es el mismo algoritmo sobre una definición de transacción distinta, elegida por la

pregunta.

3. **El tramo de monto es una discretización obligatoria.** Un algoritmo de patrones frecuentes solo entiende ítems, no puede recibir una variable continua, así que el monto se parte en cuatro categorías: menos de 50 mil, de 50 mil a 500 mil, de 500 mil a 5 millones y más de 5 millones.
4. **Se excluyen los procesos con número de postores imputado.** Si no, la asociación la podría producir nuestra propia regla de imputación de la Parte I, que sería circular.

Luego se filtran las reglas cuyo consecuente es exactamente la marca de postor único.

| Si el proceso es | Soporte | Confianza | Lift |
|---|---|---|---|
| Régimen Especial más Gobierno nacional más menos de 50 mil | 0,0176 | 1,000 | 8,06 |
| Régimen Especial más Gobierno nacional más rubro 8010 más menos de 50 mil | 0,0157 | 1,000 | 8,06 |
| Contratación Internacional más rubro 8210 | 0,0058 | 1,000 | 8,06 |
| Contratación Directa más rubro 8013 más de 50 mil a 500 mil | 0,0075 | 0,990 | 7,98 |

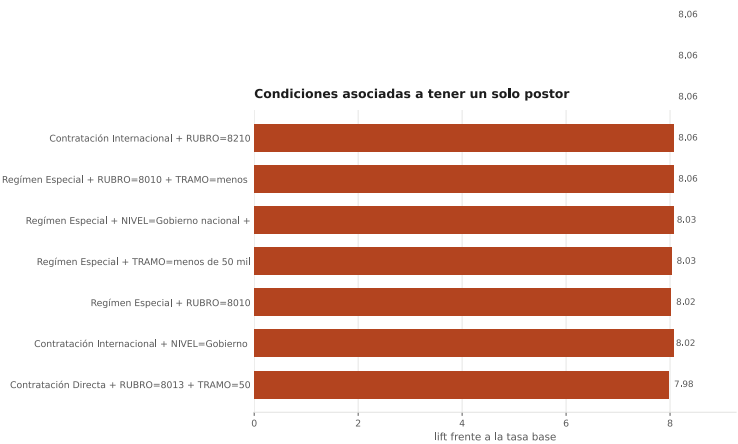
La última se lee así: cuando un proceso es Contratación Directa, del rubro 8013 y está en el tramo de 50 mil a 500 mil soles, tiene un solo postor el 99 por ciento de las veces, casi ocho veces más de lo que correspondería al azar.

**Qué son esos rubros**, resuelto con las descripciones del

propio dato. Esto es lo que convierte la figura en un hallazgo explicable y no en una lista de números:

| Rubro | Qué agrupa | Lectura de la regla |
|---|---|---|
| 8010 | Consultoría, seguimiento y monitoreo de proyectos de inversión pública | Contratación de un consultor concreto, donde el postor único es la norma del procedimiento |
| 8013 | Alquiler de inmuebles, locales y oficinas | El dueño de un local determinado es, por definición, el único que puede ofrecerlo |
| 8210 | Publicidad y difusión en medios de comunicación | Espacios de un medio concreto, que tampoco tiene sustituto |
| 5013 | Leche y derivados | El rubro del Vaso de Leche, que aparece en las reglas de productos |

La **figura f37** es la respuesta: el suelo de procesos sin competencia **no se reparte al azar**, se concentra en combinaciones identificables.




**f37.** Dónde se concentran los procesos sin competencia

**Tres matices que hay que decir al presentarla.**

1. La tasa base de postor único es del 12,4 por ciento, de modo que el lift máximo posible es 8,06 y lo alcanza **toda** condición con confianza igual a uno. Hay varias combinaciones empatadas en ese máximo, así que el ranking por lift no distingue entre ellas y se desempata por soporte.
2. Que un régimen especial, una contratación internacional o el alquiler de un inmueble tengan un solo postor es **administrativamente esperable** y no indica nada irregular. Los rubros de la tabla anterior lo muestran: son objetos sin sustituto.
3. Por eso el valor de la figura es acotar dónde buscar, y lo informativo son las combinaciones donde el procedimiento sí preveía concurrencia.

## 5. Alternativas para cada decisión, explicadas

**Para definir la transacción**

1. **Un proceso igual a una canasta.** La definición natural. Medida y descartada: 2,1 por ciento de coocurrencia, tamaño medio 0,74.
2. **Una entidad en un trimestre.** Más coocurrencia, 68 por ciento, pero menos transacciones y peor interpretabilidad.
3. **Una entidad en un año, o la entidad entera.** Llevado al extremo, la canasta acabaría siendo el catálogo completo de lo que compra una entidad, y las reglas solo dirían que las entidades grandes compran de

todo.

4. **Un proveedor en un mes.** Otra agrupación posible, que respondería qué le compran juntos a un mismo proveedor. No se usa porque la pregunta heredada de la Parte I es sobre el comportamiento de la **entidad compradora**.
5. **Lo que se hizo.** Entidad y mes, con las tres definiciones medidas y publicadas en la figura f34, incluida la que no funcionó.

**Para el nivel de agregación del código**

1. **El código CUBSO completo, 16 dígitos.** Casi único por ítem. El universo sería tan disperso que ningún conjunto alcanzaría el soporte mínimo.
2. **El prefijo de 8 dígitos.** Lo que se usó para las reglas de productos: 4.664 ítems distintos, que es un universo minable.
3. **El prefijo de 4 dígitos.** Lo que se usó en la canasta enriquecida, donde interesa el rubro grueso y no el producto exacto, para que las condiciones de la figura f37 sean legibles.

**Para hallar los conjuntos frecuentes**

1. **Fuerza bruta sobre todos los subconjuntos.** Dos elevado al número de ítems. Impensable.
2. **A-Priori.** Lo que se implementó a mano. Por niveles, con poda por antimonotonía. Su costo es una pasada completa por las transacciones en cada nivel.

3. **PCY, de Park, Chen y Yu.** La mejora clásica de A-Priori para el paso de los pares, que es el que más memoria consume. Mientras cuenta los ítems en la primera pasada, además hashea cada par a un conjunto de contenedores y cuenta cuántos pares caen en cada uno. Un par solo puede ser frecuente si su contenedor lo es, de modo que en la segunda pasada el mapa de bits de contenedores frecuentes poda muchísimos pares antes de contarlos. No hace falta aquí: con 4.664 ítems los pares caben en memoria sin apretar.
4. **Multietapa y multihash.** Son refinamientos de PCY, con más pasadas y más funciones hash respectivamente, para apretar todavía más la poda. Mismo motivo para no usarlos.
5. **Algoritmo de Toivonen.** Corre A-Priori sobre una muestra con un umbral algo menor, y luego verifica los candidatos y su **frontera negativa**, es decir los conjuntos que no son frecuentes en la muestra pero cuyos subconjuntos sí lo son. Si ningún elemento de la frontera negativa resulta frecuente en el conjunto completo, el resultado es exacto; si alguno lo es, hay que repetir. Da una respuesta exacta con alta probabilidad en dos pasadas. No se usa porque el conjunto completo cabe en memoria: lo que escala mal aquí es A-Priori, no el dato.
6. **SON, de Savasere, Omiecinski y Navathe.** Es el algoritmo distribuido natural y conviene conocerlo justo porque esta parte usa Spark. Parte las transacciones en fragmentos, halla los conjuntos frecuentes locales de cada fragmento con cualquier algoritmo, y toma la unión como candidatos globales; después una

segunda pasada cuenta esos candidatos sobre el conjunto completo. La propiedad que lo hace correcto es que un conjunto frecuente globalmente tiene que ser frecuente en al menos un fragmento, de modo que **no hay falsos negativos**. Encaja exactamente en un mapa y una reducción, y es la forma en que se distribuiría A-Priori si hiciera falta. No se usa porque FP-Growth de MLlib ya resuelve el conjunto completo en menos de medio segundo.

7. **Eclat.** Representa cada ítem por su lista de identificadores de transacción y obtiene el soporte de un conjunto intersecando listas, en lugar de recorrer las transacciones. Es rápido con datos densos y consume mucha memoria con universos grandes.
8. **FP-Growth.** Lo que se usa en producción. Árbol de prefijos y proyecciones condicionales, sin generar candidatos.

**Para elegir qué reglas presentar**

1. **Confianza alta.** Descartada: no descuenta la frecuencia del consecuente, de modo que una regla que concluye en un ítem muy común tiene confianza alta por construcción.
2. **Lift.** Lo que se usó, junto a un mínimo de soporte para que la regla no sea una anécdota.
3. **Convicción.** Mide cuánto se equivocaría la regla si antecedente y consecuente fueran independientes. Es menos conocida y no añade nada aquí.
4. **Leverage, o diferencia respecto de la**

**independencia.** Favorece reglas de soporte alto, que es lo contrario de lo que hace el lift. Serviría si la prioridad fuera el impacto absoluto y no la fuerza del patrón.

5. **Conjuntos maximales y cerrados.** Formas de comprimir la salida: los maximales son los frecuentes sin superconjunto frecuente, y los cerrados son aquellos cuyos superconjuntos tienen soporte estrictamente menor. Reducen mucho el número de resultados sin perder información de soporte en el caso cerrado. No hacían falta porque la salida ya es manejable.

**Para responder la pregunta de la Parte I**

1. **Un modelo de clasificación de postor único.** Predeciría mejor, y no respondería la pregunta: el enunciado pide patrones frecuentes, y un modelo entrega un peso por variable en lugar de combinaciones concretas legibles.
2. **Tablas cruzadas de método por nivel por tramo.** Es lo que haría un análisis exploratorio, y se queda corto: el número de combinaciones crece y no hay criterio para ordenarlas.
3. **Lo que se hizo.** Convertir los atributos del proceso en ítems de la canasta, de modo que el mismo algoritmo de patrones frecuentes descubra asociaciones entre atributos, con soporte, confianza y lift como criterio de orden.

**Lo que no se hizo y sería la continuación natural**

1. **Minería de secuencias, al estilo GSP o PrefixSpan.** Encuentra patrones **con orden**, por ejemplo que tras comprar A una entidad compra B el mes siguiente. Nuestra pregunta es de coocurrencia dentro de un mes, no de orden, pero es la extensión evidente si se quisiera estudiar ciclos de abastecimiento.
2. **Reglas con restricción temporal o por territorio.** Mismo algoritmo, particionando antes.

## 6. Preguntas probables y qué responder

**¿No es hacer trampa cambiar la definición de transacción hasta que salga algo?** Sería trampa si la nueva no tuviera sentido. Lo tiene, y la figura documenta las tres definiciones para que el lector juzgue el cambio.

**¿Por qué implementar A-Priori si FP-Growth gana?** Porque A-Priori es donde se ve la antimonotonía, y porque la figura es el argumento medido en lugar de la cita del libro.

**¿Por qué no filtrar simplemente por confianza alta?** Porque la confianza no descuenta la frecuencia del consecuente, y el lift sí.

**¿Eso es evidencia de direccionamiento?** No. Parte de lo que aparece arriba es administrativamente esperable. Lo que entrega el análisis es una lista priorizada de dónde mirar, no una conclusión sobre conducta.

**¿Cómo saben que su A-Priori es correcto?** Porque devuelve exactamente el mismo conjunto de itemsets

frecuentes que FP-Growth de MLlib sobre las mismas transacciones, con diferencia de soporte nula.

# Glosario del dato

Qué significa cada campo, cada código y cada etiqueta del dataset de contrataciones públicas del Perú que usa el proyecto. Está pensado para la sustentación: si en la presentación aparece un rubro 8013 o un Régimen Especial, aquí está qué es y de dónde salió.

Las cifras son las de la tabla analítica construida en la Parte I, es decir 177.398 procesos de 2023 a 2025. El porqué de cada decisión de limpieza está en JUSTIFICACION.md y el recorrido del notebook en notebooks/1_kdd_eda.md.

## 1. De dónde viene el dato

1. **OCDS**, Open Contracting Data Standard. Es un estándar internacional para publicar contratación pública con una estructura común, de modo que un mismo análisis pueda correr sobre países distintos. El Perú publica sus contrataciones en este formato.
2. **Paquete anual.** La descarga viene en un paquete por año, y cada paquete son **21 tablas CSV** que son la versión plana del documento OCDS, una por nivel de anidamiento. El proyecto usa **5 de las 21**, lo justo para el grano de proceso.
3. **ocid**, Open Contracting Identifier. Es la llave del proceso de contratación, única y estable a lo largo de todas las tablas. Se ve así: ocds-dgv273-seacev3-1000028. Hay 177.398 valores distintos y ninguno repetido, porque el grano de la tabla analítica es exactamente un proceso.

4. **SEACE** es el Sistema Electrónico de Contrataciones del Estado, la plataforma donde las entidades publican. El prefijo seacev3 del ocid viene de ahí.

**Las cinco tablas que se usan y para qué:**

| Tabla cruda | Qué aporta |
|---|---|
| main.csv | El proceso: fechas, objeto, método, categoría, comprador, monto y número de postores |
| awards.csv | El monto de cada adjudicación, usado para quedarse con la mayor cuando hay varias |
| awards_suppliers.csv | Quién ganó, es decir el proveedor, y cuántos proveedores distintos hubo |
| awards_items.csv | Los ítems adjudicados con su código CUBSO y su descripción, que es la canasta de la Parte V |
| parties.csv | Los participantes y su rol, de donde sale el departamento de la entidad compradora |

## 2. Los actores, que es la confusión más común

1. **Entidad, o entidad compradora.** El Estado que compra. En el dato es el campo **comprador_id** y hay 2.968 distintas. Puede ser una municipalidad distrital, un gobierno regional, un hospital, una UGEL, un ministerio, una universidad pública, una empresa del Estado o un programa social.
2. **Postor.** La empresa o persona que se presenta a competir en un proceso. Lo que se cuenta en **n_postores**.
3. **Proveedor, o adjudicatario.** El postor que ganó. En el dato es **proveedor_id** y hay 67.510 distintos.

La regla para no equivocarse: **postor es el que compite, proveedor es el que ganó, entidad es la que paga.** Todo postor es una empresa; la entidad nunca lo es, aunque haya empresas del Estado entre los compradores.

## 3. El ciclo de un proceso y qué significa que falte algo

1. **Convocatoria.** La entidad publica el proceso con su objeto, su valor estimado y su método. La fecha de esa publicación es el campo **fecha**.
2. **Presentación de ofertas.** Se presentan los postores, y su número queda en n_postores.
3. **Adjudicación.** Se elige al ganador, que queda en proveedor_id. Un proceso puede tener **varias adjudicaciones**, por ejemplo un proceso por ítems en el que cada lote lo gana una empresa distinta. Se guarda el proveedor de la adjudicación de mayor monto y aparte el conteo, porque tener varios proveedores en un mismo proceso es una señal de fraccionamiento que conviene medir. Ocurre en 6.342 procesos.
4. **Un proceso sin adjudicación** quedó desierto, fue anulado o seguía en trámite cuando se publicó el paquete. Son 31.373, el 17,7 por ciento. No es un error del dato, así que el proyecto lo **marca** con el campo adjudicado en lugar de borrarlo.

## 4. Los campos de la tabla analítica

| Campo | Qué es | Detalle que conviene saber |
|---|---|---|
| ocid | Llave del proceso | Única, 177.398 valores |
| fecha | Cuándo se convocó el proceso | Es tender_datePublished, el evento, no el registro |
| fecha_ registro | Cuándo lo publicó el sistema | Es publishedDate. Se separa a propósito porque el paquete anual republica procesos convocados años antes, y usar esta fecha describiría la actividad del publicador y no la del Estado comprador |
| objeto | La descripción libre del proceso | 237.298 valores distintos en el dato crudo. Es el texto sobre el que trabajan las Partes II y III |
| categoria | goods, services o works | Bienes 79.287, servicios 78.135, obras 19.976 |
| metodo | El procedimiento de selección | 40 valores distintos, dominados por doce |
| comprador_id, comprador_ nombre | La entidad compradora | 2.968 entidades |
| nivel_ gobierno | Nivel al que pertenece la entidad | Campo **derivado**, no viene en el dato. Ver la sección 7 |
| departamento | Departamento de la entidad | Sale de parties.csv. Es dónde está la entidad, **no** dónde se ejecuta la obra |
| monto_ pen | Valor del proceso en soles | Es el **valor referencial o estimado de la convocatoria**, no lo que finalmente se pagó. Mediana 178 mil soles, máximo 1.135 millones |
| moneda | Moneda original | 98,2 por ciento en soles; el resto se convierte |
| n | Cuántos | Mediana 9, máximo 107 tras el recorte |

## 5. El código CUBSO, dígito por dígito

**CUBSO** es el Catálogo Único de Bienes, Servicios y Obras, el catálogo oficial con el que el Estado peruano nombra lo que contrata. Es importante entender que **incluye servicios y obras y no solo bienes**, porque de ahí salen ítems como servicio de alquiler de excavadora hidráulica.

El código tiene **16 dígitos** y es jerárquico:

| Tramo | Qué identifica | Ejemplo con 8013150200094959 |
|---|---|---|
| Dígitos 1 y 2 | Segmento, la gran familia económica | 80, servicios de gestión y administración |
| Dígitos 3 y 4 | Familia dentro del segmento | 13, alquiler de inmuebles |
| Dígitos 5 a 8 | Clase y artículo, el bien o servicio concreto | 1502, servicio de alquiler de inmueble |
| Dígitos 9 a 16 | Correlativo de la ficha concreta del catálogo | 00094959 |

**Los tres niveles que usa el proyecto y por qué cada uno:**

1. **16 dígitos, el código completo.** No se usa. Es casi único por ítem, de modo que el universo sería tan disperso que ningún conjunto alcanzaría el soporte mínimo en la Parte V.
2. **8 dígitos, el prefijo de catálogo.** Es el nivel del campo items y el que se usa para las reglas de productos. Da 4.664 ítems distintos en la tabla analítica, que es un universo minable.

3. **4 dígitos, el rubro.** Es segmento más familia. Se usa en la canasta enriquecida, donde interesa el sector grueso y no el producto exacto, para que las condiciones salgan legibles. Da 368 rubros, repartidos en 56 segmentos.

**Los segmentos más frecuentes**, obtenidos contando las descripciones del propio dato:

| Segmento | Qué agrupa | Ítem más frecuente |
|---|---|---|
| 72 | Construcción y mantenimiento de infraestructura | Servicio de mantenimiento rutinario de caminos |
| 81 | Servicios de ingeniería y supervisión | Supervisión de obra de construcción de calles |
| 80 | Servicios de gestión, consultoría y alquileres | Servicio de seguimiento y monitoreo de actividades de campo |
| 30 | Materiales de construcción | Cemento Portland puzolánico tipo IP de 42,5 kg |
| 42 | Equipamiento y material médico | Respirador de polipropileno tipo N-95 |
| 50 | Alimentos y bebidas | Leche evaporada entera de 400 g |
| 15 | Combustibles | Diésel B5 S-50 |
| 51 | Medicamentos | Cloruro de sodio al 0,9 por ciento inyectable |
| 41 | Laboratorio y medición | Hemograma automatizado diferencial |
| 78 | Transporte y alquiler de vehículos | Servicio de alquiler de camión volquete |
| 25 | Vehículos | Camioneta pick up cabina doble 4 por 4 |
| 82 | Publicidad, diseño e impresión | Servicio de publicidad en página web |

**Los cuatro rubros que aparecen en los resultados de la Parte V:**

| Rubro | Qué agrupa | Por qué aparece |
|---|---|---|
| 8010 | Consultoría y monitoreo de proyectos de inversión pública | Es el rubro de las condiciones de postor único con Régimen Especial |
| 8013 | Alquiler de inmuebles, locales y oficinas | Postor único con Contratación Directa: el dueño de un local es el único que puede ofrecerlo |
| 8210 | Publicidad y difusión en medios | Postor único con Contratación Internacional |
| 5013 | Leche y derivados | El rubro del Vaso de Leche, que aparece en las reglas de productos |

## 6. Los métodos de contratación

El método es el procedimiento de selección, y es el campo que dice **cuánta competencia preveía la norma**. Por eso es la variable central de la pregunta sobre postor único: comparar el número de postores entre métodos sin tener esto en cuenta es comparar peras con manzanas.

| Método | Procesos | Monto, miles de millones de soles | Postores en promedio | Para qué sirve |
|---|---|---|---|---|
| Adjudicación Simplificada | 82.503 | 47,1 | 16,1 | El procedimiento ordinario de monto intermedio, el más usado con diferencia |
| Subasta Inversa Electrónica | 21.150 | 10,0 | 8,2 | Bienes y servicios estandarizados con ficha de catálogo, donde solo se compite por precio |
| Comparación de Precios | 12.555 | 0,9 | 3,1 | Bienes y servicios de disponibilidad inmediata y bajo monto |
| Contratación Directa | 11.049 | 14,5 | 1,2 | Excepcional y con causal: desabastecimiento, emergencia, proveedor único o servicio personalísimo |
| Licitación Pública Abreviada | 9.601 | 6,3 | 13,6 | Versión abreviada de la licitación, solo en 2025 |
| Licitación Pública | 9.242 | 106,6 | 21,0 | Bienes y obras de mayor monto. Es el 5 por ciento de los procesos y el mayor monto del dataset |
| Concurso Público Abrevia- | 8.926 | 1,9 | 14,6 | Versión abreviada del concurso, solo |

**Un hallazgo del propio dato que conviene saber explicar.** Licitación Pública Abreviada, Concurso Público Abreviado y Concurso Público de Servicios tienen **cero procesos en 2023 y 2024 y aparecen enteros en 2025**, con 19.501 procesos entre los tres. Al mismo tiempo la Adjudicación Simplificada cae de 33.201 en 2024 a 12.919 en 2025. Es decir que **hay un cambio de régimen normativo dentro de la ventana de análisis**, el paso a la nueva Ley General de Contrataciones Públicas, y la serie por método no es comparable de punta a punta. Cualquier conclusión que mezcle métodos a lo largo de los tres años tiene que decirlo.

## 7. Los niveles de gobierno

**Es un campo derivado, no viene en el dato.** Se construye con una regla sobre el nombre de la entidad, porque OCDS no publica el nivel de gobierno y sin él no se puede comparar una municipalidad con un ministerio. Conviene declararlo abiertamente, porque es una decisión del equipo y tiene un margen de error conocido.

| Nivel | Procesos | Cómo se reconoce |
|---|---|---|
| Gobierno local | 73.456 | El nombre contiene municipalidad o municipio |
| Gobierno regional | 39.776 | Gobierno regional, dirección regional, gerencia regional |
| Gobierno nacional | 29.984 | Ministerio, instituto, superintendencia, organismo, programa, y otros nombres de entidades nacionales |
| Otro | 10.742 | No encaja en ninguna regla, el 6,1 por ciento |
| Empresa del Estado | 8.048 | Sociedad anónima, empresa, entidad prestadora de servicios, Electro, Sedapal, Banco de la Nación |
| Universidad pública | 7.094 | El nombre contiene universidad |
| Fuerzas armadas y policiales | 5.812 | Ejército, Marina, Fuerza Aérea, Policía, ministerios de Defensa e Interior |
| Establecimiento de salud | 2.278 | Hospital, red de salud, red asistencial, Diresa |
| Unidad educativa | 208 | Unidad de gestión educativa local o UGEL |

Las dos cosas que hay que decir si preguntan: que el 6,1 por ciento cae en Otro y que la clasificación es por nombre, de modo que una entidad con un nombre atípico puede quedar mal ubicada. Es suficiente para agregar y comparar, y no lo sería para una afirmación sobre una entidad individual.

## 8. La categoría y el tramo de monto

**Categoría** es el campo de OCDS mainProcurementCategory y tiene tres valores: goods

para bienes, services para servicios y works para obras. Están casi empatados en número de procesos, pero no en monto: las obras son el 11 por ciento de los procesos y concentran mucho más dinero por proceso.

**Tramo de monto** no es un campo del dato, es una discretización que el proyecto construye donde hace falta. Se usa en dos sitios con cortes distintos y a propósito:

1. En la Parte I, seis tramos, para describir la competencia a lo largo de toda la escala: menos de 50 mil, 50 a 200 mil, 200 mil a 1 millón, 1 a 5 millones, 5 a 20 millones y más de 20 millones.
2. En la canasta enriquecida de la Parte V, cuatro tramos, porque un algoritmo de patrones frecuentes solo entiende ítems y no puede recibir una variable continua: menos de 50 mil, 50 mil a 500 mil, 500 mil a 5 millones y más de 5 millones.

El monto se trabaja **en escala logarítmica** en casi todas las figuras. No es un adorno: va de cuatro céntimos a 1.135 millones de soles, así que en escala lineal todo el dataset se apelmaza contra el eje y solo se ven los extremos.

## 9. Las marcas y los indicadores derivados

1. **Postor único**, n_postores menor o igual a uno. Es el indicador central del proyecto y la tasa base es del **12,4 por ciento** de los procesos. Atraviesa todos los tramos de monto, y en el tramo de menos de 50 mil soles llega al 57 por ciento. Esa es la pregunta que la Parte I deja abierta y que la Parte V responde.

2. **n_postores_imputado.** El número de postores faltaba en el 8,3 por ciento del dato crudo, y en la tabla final se imputó en 6.655 procesos con la mediana de su propio método, no con la mediana global, porque la competencia depende del procedimiento. La marca existe para poder **excluir esos procesos** cuando el análisis es sobre la competencia misma, y así evitar que la propia imputación genere el patrón. La Parte V la usa exactamente para eso.
3. **monto_outlier.** Se marca con el criterio de Tukey sobre el logaritmo del monto y factor 3, lo que da un rango de 584 soles a 65 millones. Quedan fuera 565 procesos, el 0,3 por ciento, y **se conservan**: un contrato de mil millones de soles no es un error de digitación, es la obra grande del año, y borrarlo sería borrar el objeto de estudio.
4. **Gini del monto, 0,857.** Mide la desigualdad en el reparto del dinero. Cero sería reparto perfectamente igual y uno sería que todo el monto está en un solo proceso. El 1 por ciento de los procesos concentra el 47,1 por ciento del monto, y el 10 por ciento concentra el 80,2 por ciento. Es la curva de Lorenz de la Parte I.
5. **Fraccionamiento.** No se mide directamente, pero n_ proveedores mayor que uno y la coocurrencia mensual de la Parte V son las dos señales que lo rodean.

## 10. Confusiones frecuentes y cómo responderlas

1. **El monto no es lo que se pagó.** monto_pen es el valor referencial de la convocatoria. El monto adjudicado existe en awards.csv y se usa para elegir la

adjudicación principal, pero la cifra que recorre las figuras es la del proceso. Decir que el Estado gastó tanto es impreciso; lo correcto es que convocó procesos por ese valor.

2. **Un proceso no es un contrato.** Un proceso puede terminar en varios contratos, quedar desierto o ser anulado. El grano del proyecto es el proceso.
3. **El departamento es el de la entidad.** Un ministerio con sede en Lima que financia una obra en Puno aparece en Lima. Por eso Lima tiene el 30 por ciento de los procesos.
4. **La fecha es la de convocatoria, no la de publicación del paquete.** Son dos campos distintos y la diferencia importa: 81.234 filas del dato crudo quedan fuera de la ventana de 2023 a 2025, casi todas por ser anteriores, y se descartan para que la serie temporal mida actividad y no republicación.
5. **Postor único no es irregularidad.** En Contratación Directa, Régimen Especial o Contratación Internacional es administrativamente esperable, y en el alquiler de un inmueble es inevitable. El análisis entrega una lista priorizada de dónde mirar, no una conclusión sobre conducta.
6. **Nada de esto es una afirmación sobre una entidad o un proveedor concreto.** Son asociaciones estadísticas sobre datos de publicación. En contratación pública esa distinción no es un formalismo: confundirla convierte un ejercicio de minería en una acusación sin sustento.

## 11. Cifras que conviene tener en la cabeza

| Cifra | Valor |
|---|---|
| Procesos en la tabla analítica | 177.398 |
| Período | 2023 a 2025 |
| Entidades compradoras distintas | 2.968 |
| Proveedores distintos | 67.510 |
| Ítems CUBSO distintos, a 8 dígitos | 4.664 |
| Rubros distintos, a 4 dígitos | 368 |
| Monto mediano de un proceso | 178 mil soles |
| Monto máximo de un proceso | 1.135 millones de soles |
| Postores en la mediana | 9 |
| Procesos con un solo postor | 12,4 por ciento |
| Procesos sin adjudicación | 31.373, el 17,7 por ciento |
| Gini del monto | 0,857 |
| Tablas OCDS usadas, de 21 | 5 |